



2024 - 2025

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : INFORMATION TECHNOLOGY

**An AI-Driven Legal Assistance System Based on a
GraphRAG-Oriented Multi-Agent Architecture**

By: Dhikra BEN MAHMOUD

Academic supervisor: Mrs.Nardine HANFI

Corporate Internship Supervisor: Mrs.Chayma BARKAOU



I validate the submission of the student's report:

1. Name & Surname : Dhikra BEN MAHMOUD

Business site Supervisor

2. Name & Surname : Mrs.Chayma BARKAOUI

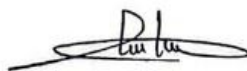
Cachet & Signature / Stamp & Signature



Academic Supervisor

3. Name & Surname : Mrs.Nardine HANFI

Signature / Signature



Dedication

I praise Allah (Alhamdulillah Rabbil Alameen) for granting me the strength, wisdom, and perseverance to complete this work, and I humbly dedicate this achievement to those who have been my pillars of support throughout this journey.

To my beloved mother **Chema**, whose endless love, prayers, and nurturing spirit have been my sanctuary in times of struggle. Her gentle guidance and unwavering confidence in my potential have illuminated my path when darkness seemed overwhelming. Through her sacrifices and boundless devotion, she taught me that true strength lies in compassion and resilience.

To my dear father **Ferjani**, whose wisdom, integrity, and steadfast support have been the foundation upon which I built my dreams. His quiet strength and encouraging words gave me the courage to pursue excellence and the determination to overcome every challenge. His belief in me became the driving force behind my perseverance.

To my cherished sisters **Hanen** and **Ayda**, whose laughter, care, and unwavering support have filled my life with joy and motivation. Their presence has been a constant reminder that success is sweeter when shared with those who truly matter. Through their encouragement, they showed me that family bonds transcend all obstacles.

To my beloved brother **Mourad**, whose companionship, understanding, and brotherly love have been a source of comfort and strength. His support and shared dreams have made this journey less solitary and more meaningful, reminding me that we rise together as a family.

To all my family members, whose prayers, love, and patience have sustained me through the most demanding moments of this endeavor. Their collective faith in my abilities transformed my doubts into determination and my fears into courage.

And to my dear friends, who have been my companions through this academic journey, offering their time, encouragement, and genuine support. Their friendship turned challenges into opportunities for growth and made every milestone worth celebrating.

This accomplishment belongs not only to me but to each of you who believed in me when I struggled to believe in myself. Your love, prayers, and unwavering support have made this dream a reality. May Allah bless you all as you have blessed my life.

Dhikra Ben Mahmoud

Acknowledgements

I would like to begin by expressing my profound gratitude to my academic supervisor, **Mrs. Nardine Hanfi**, for her exceptional mentorship, dedicated guidance, and thoughtful support throughout the entire duration of this project. Her scholarly expertise, meticulous attention to detail, and continuous availability have been pivotal in shaping this research. Her constructive critiques and academic excellence have elevated the standard of this work beyond my initial expectations.

My sincere appreciation extends to **Mrs. Chayma Barkaoui**, my professional supervisor at E-TAFAKNA, for her warm reception into the organization and her confidence in my capabilities. Her professional wisdom, strategic insights, and consistent encouragement have transformed this internship into an invaluable learning experience that has significantly contributed to both my personal growth and the successful completion of this project.

I am deeply grateful to the entire **E-TAFAKNA** team for creating a dynamic and supportive work environment. Their collective expertise, collaborative approach, and generous sharing of knowledge have made this complex project not only achievable but also genuinely rewarding. Their professional camaraderie and technical assistance have been indispensable throughout this journey.

My recognition also goes to the distinguished faculty members and dedicated administrative staff at **ESPRIT** for their commitment to academic excellence and their role in nurturing future engineers. Their rigorous educational approach and unwavering support have equipped us with the skills and knowledge necessary to excel in our professional endeavors.

Lastly, I wish to thank all those who have contributed, whether directly or indirectly, to the realization of this work. Your encouragement, feedback, and faith in this project have been fundamental to achieving this academic and professional milestone.

List of Abbreviations

ACR	= Azure Container Registry
AI	= Artificial Intelligence
API	= Application Programming Interface
AVX2	= Advanced Vector Extensions 2
CI/CD	= Continuous Integration/Continuous Deployment
CPU	= Central Processing Unit
CRISP-DM	= Cross-Industry Standard Process for Data Mining
FAISS	= Facebook AI Similarity Search
Firebase	= Google's backend-as-a-service platform
Flask	= Python web framework
GPU	= Graphics Processing Unit
GPT-3.5	= Generative Pre-trained Transformer 3.5
GPT-4	= Generative Pre-trained Transformer 4
GPT-4o	= Optimized version of GPT-4
HiiL	= The Hague Institute for Innovation of Law
HNSW	= Hierarchical Navigable Small World
ICE	= Interactive Connectivity Establishment
KEDA	= Kubernetes Event-Driven Autoscaler
KPI	= Key Performance Indicator
LLM	= Large Language Model
Llama	= Large Language Model Meta AI
MENA	= Middle East and North Africa
ML	= Machine Learning
NDA	= Non-Disclosure Agreement
Neo4j	= Graph database platform
NLP	= Natural Language Processing
OAuth2	= Open Authorization 2.0
OCI	= Open Container Initiative
PKL	= Pickle Files
QA	= Question Answering
RAG	= Retrieval-Augmented Generation
RBAC	= Role-Based Access Control
React	= JavaScript library for user interfaces
REST	= Representational State Transfer
SDK	= Software Development Kit
SME	= Small and Medium Enterprises
SSML	= Speech Synthesis Markup Language
StartUp'Act	= Tunisian startup support program
STT	= Speech-to-Text
SUVAS	= AI-powered translation platform (India)
TDSP	= Team Data Science Process
TLS	= Transport Layer Security
TTS	= Text-to-Speech
TypeScript	= Programming language
UUID	= Universally Unique Identifier
VAD	= Voice Activity Detection
WebRTC	= Web Real-Time Communication
WSGI	= Web Server Gateway Interface

Table of Contents

GENERAL INTRODUCTION	1
CHAPTER I: PROJECT GENERAL FRAMEWORK	3
1. INTRODUCTION	4
2. HOST ORGANIZATION PRESENTATION	4
3. GENERAL PROJECT OVERVIEW	4
3.1 Project Framework.....	4
3.2 Study of Existing Solution	5
3.3 Critical of the Existing System	5
3.4 Problem Statement	6
3.5 Proposed Solution	6
4. ADOPTED METHODOLOGY	7
4.1 Methodological Framework Selection	7
4.2 Comparative Analysis: CRISP-DM vs TDSP.....	7
4.3 TDSP Methodology Selection	8
5. HIERARCHICAL DIRECTORY STRUCTURE	10
5.1 Introduction to Practical Architecture.....	12
5.2 Documentation Directory (docs/)	12
5.3 Data Directory (data/)	12
5.4 Data Directory (data/)	13
5.5 Integration Architecture Benefits.....	14
6. CHOICE OF SERVICES, TECHNOLOGIES, AND TOOLS	14
6.1 Azure AI Search: Core Infrastructure Foundation.....	14
6.2 Tunisia Legal Index Implementation	15
6.3 Resource Allocation and Management	15
6.3.1 Service Configuration	15
6.3.2 Integration Architecture	16
6.4 Development Environment and Programming Languages	16
6.4.1 Visual Studio Code.....	16
6.4.2 Python.....	17
6.4.3 JavaScript/HTML/CSS.....	17
6.4.1 React.....	18
6.5 Backend Development Framework.....	19
6.6 Azure Cloud Services Integration	19
6.6.1 Azure Cognitive Services Speech	19
6.6.2 Azure OpenAI	20
6.6.3 Azure AI Foundry	20
6.6.4 Azure Identity.....	21
6.7 Authentication and Database Services.....	22
6.8 Technology Stack Architecture.....	22
6.8.1 Integrated Development Workflow.....	22
6.8.2 Development Environment Integration	23
7. CONCLUSION.....	23

CHAPTER II: BUSINESS UNDERSTANDING	24
1. INTRODUCTION	25
2. BUSINESS UNDERSTANDING	25
2.1 Business Objectives	25
2.1.1 Primary Goal :	25
2.1.2 Strategic Objectives:	25
2.2 Project Team Structure	26
2.3 End User Communities	26
2.4 Comprehensive Business Requirements	26
2.4.1 Functional Requirements Analysis.....	26
2.4.2 Non-Functional Requirements Framework.....	27
3. SOLUTION APPROACH OVERVIEW.....	28
3.1 Business Process Steps	28
3.2 Technical Terms Explained	29
4. CONCLUSION	30
CHAPTER III: DATA ACQUISITION & UNDERSTANDING	31
1. INTRODUCTION	32
2. DATA ACQUISITION & UNDERSTANDING	32
2.1 Data Acquisition	32
2.2 Data Understanding:	32
2.2.1 Structural Analysis	32
2.2.2 Exploratory Data Analysis (EDA)	33
2.2.3 System Suitability	34
3. CONCLUSION	34
CHAPTER IV: MODELING.....	35
1. INTRODUCTION	36
2. MODELING.....	36
2.1 Initial RAG Implementation	36
2.2 Evaluation of Initial Implementation	37
2.2.1 Performance Analysis & Critical Issues.....	37
2.2.2 Team Project Meeting & Strategy Revision	37
2.3 Solution Architecture Overview	38
2.4 Hierarchical Chunking & Embedding Implementation	39
2.4.1 Advanced Embedding Generation ProcessFollowing hierarchical extraction,	39
2.4.2 Vector Search Infrastructure: "tunisialegalindex"	39
2.5 RAG Architecture Comparative Study	40
2.5.1 Advanced Embedding Generation Process	41
2.5.2 Vector RAG Implementation	42
2.5.3 Comparative Analysis and Decision	42
2.6 System Design and Technical Implementation Process	43
2.6.1 System Overview and Process Foundation	43
2.6.2 User Authentication and Session Management Process	44
2.6.3 Real-Time Voice Interaction Pipeline.....	44
2.6.4 Flexible Text-Based Interaction Alternative	46

2.6.5 Advanced Technical Optimizations and Performance Engineering	47
2.6.6 Production Deployment.....	47
2.7 Azure AI Foundry: Multi-Agent System Implementation.....	48
2.7.1 Platform Overview	48
2.7.2 Multi-Agent Architecture Design.....	49
2.7.3. Question-Answer Agent Implementation.....	50
2.7.4 Reviewer Agent Implementation	50
2.8 Azure Speech Services Implementation	51
2.8.1 Speech-to-Text (STT) Service	51
2.8.2 Text-to-Speech (TTS) Service	51
2.8.3 Animated Avatar Service	52
2.9 Evaluation	53
3. CONCLUSION.....	54
CHAPTER V: DEPLOYMENT	55
1. INTRODUCTION	56
2. DEPLOYMENT	56
2.1 Deployment Phase Overview.....	56
2.2 User Registration Interface	56
2.3 User Login Interface	57
2.4 AI Legal Consultation Interface.....	57
2.5 Technical Implementation Impact	58
2.6. Data Visualization Overview	58
2.6.1 Dashboard Visualizations.....	62
2.7. Complete Application Deployment	63
2.7.1 Tools Definition	63
2.7.2 Deployment Process.....	65
3. CONCLUSION	67
GENERAL CONCLUSION.....	68
WEBOGRAPHY	70
APPENDIX A: VAD (VOICE ACTIVITY DETECTION) IMPLEMENTATION CODE	72

List of Tables

Table 1. 1. Comparative Analysis of CRISP-DM and TDSP Methodologies	7
Table 2. 1: Project Team Roles and Responsibilities.....	26
Table 5. 1: Core Business Metrics and User Engagement Indicators	59
Table 5. 2 : Real-time System Performance and Activity Indicators.....	60
Table 5. 3: Daily Message Volume Trend Analysis	60
Table 5. 4: User Activity Distribution and Interaction Patterns.....	60
Table 5. 5: Hourly Platform Usage and Peak Time Analysis	61
Table 5. 6: Multimodal Interaction Feature Adoption Tracking.....	61
Table 5. 7: Real-time User Activity and System Event Monitoring	61

List of Figures

Figure 1. 1: Logo E-tafakna	4
Figure 1. 2: TDSP Iterative Methodology	9
Figure 1. 3: Azure AI Search Service	15
Figure 1. 4: Visual Studio Code Logo.....	16
Figure 1. 5: Python Programming Language Logo.....	17
Figure 1. 6: Web Technologies Stack Logo	17
Figure 1. 7: React Framework Logo	18
Figure 1. 8: Flask Framework Logo.....	19
Figure 1. 9: Azure Cognitive Services Speech Logo	19
Figure 1. 10: Azure OpenAI Service Logo	20
Figure 1. 11: Azure AI Foundry Logo	20
Figure 1. 12: Azure Identity Logo.....	21
Figure 1. 13: Firebase Logo	22
Figure 2. 1: Legal AI Avatar Solution Architecture (Business Process Flow)	28
Figure 3. 1: Example of hierarchical structure in Tunisian legal documents.....	33
Figure 4. 1: Multi-Agent Legal RAG Architecture.....	36
Figure 4. 2: Multi-Agent Legal RAG Architecture.....	38
Figure 4. 3: Azure AI Search Index Configuration for Tunisian Legal Documents.....	39
Figure 4. 4: Neo4j Browser Graph Visualization.....	41
Figure 4. 5: Neo4j Browser Graph Visualization.....	41
Figure 4. 6 : Azure AI Foundry Multi-Agent Configuration	42
Figure 4. 7: Complete Application Process Flow - Real-Time AI Voice Agent System.....	43
Figure 4. 8 : Azure AI Foundry Agent Creation	49
Figure 4. 9: Animated Avatar Visual Representation	52
Figure 5. 1: User Account Creation Interface	56
Figure 5. 2: User Authentication Interface.....	57
Figure 5. 3 AI Legal Assistant Main Interface with Avatar Technology.....	58
Figure 5. 4: Main Dashboard Overview with Core KPIs	62
Figure 5. 5: Daily Trends and Activity Distribution Analytics.....	62
Figure 5. 6: Usage Patterns and Communication Preferences Analysis	63
Figure 5. 7: Logo GitHub Actions	63
Figure 5. 8: Logo Docker Containerization	64
Figure 5. 9 Logo Azure Container Registry Service.....	64
Figure 5. 10: Logo Azure Container Registry Service.....	65
Figure 5. 11: Azure Container Apps Configuration Interface in Microsoft Azure Portal	67

GENERAL INTRODUCTION

Imagine a lawyer in the early 1990s leafing through dusty legal volumes spending hours searching for a relevant court ruling. Today, that same task is accomplished in seconds on digital platforms such as Doctrine or Dalloz. This contrast highlights the tremendous impact of digital transformation on legal practice. Today, a deeper shift is underway: the rise of conversational artificial intelligence.

Across the world, AI is increasingly integrated into legal ecosystems. In Singapore, user-oriented chatbots have emerged within the justice framework, offering citizens access to procedural and administrative information while clearly distinguishing from legal advice. In Belgium, digital services help guide users toward the right procedures. In the Netherlands, virtual mediation tools are trialed to resolve minor disputes without going to court. In India, the SUVAS system—an AI-powered translation platform—transforms legal documents into ten vernacular languages to enhance accessibility.

Tunisia, like many other countries, is navigating this global transformation within the particularities of its own legal and institutional environment. Over the past decade, significant legislative activity and structural reforms have shaped the legal landscape. However, this dynamism has also introduced increasing levels of complexity, making it difficult for many to access and interpret legal information effectively. Entrepreneurs, citizens, and legal professionals often face obstacles such as scattered regulations, overlapping procedures, and inconsistent structures across legal texts. These hurdles translate into tangible costs—whether time, reliance on intermediaries, or misinterpretations—which limit the reach of justice and hinder informed decision-making.

In this evolving and multifaceted context, an essential question arises: how can modern technological and scientific methods help make legal knowledge more accessible, transparent, and actionable for all Tunisians?

This project addresses these challenges by developing a revolutionary Legal AI Avatar system that integrates advanced artificial intelligence capabilities, including 3D avatar visualization, multilingual voice interaction, and specialized Tunisian legal intelligence, to transform traditional legal consultation into an accessible, engaging, and highly accurate service. The approach combines cutting-edge AI technologies—such as vector search and multi-agent systems—with human-centered design principles like intuitive interfaces and real-time analytics. By leveraging Azure cloud infrastructure and following systematic data science methodologies, the solution ensures scalability, reliability, and professional-grade accuracy while maintaining the trust and accessibility required for legal services.

The report is structured into five key chapters:

- **Project General Framework:** Establishes the foundation by presenting E-TAFAKNA as the host organization, analyzing the limitations of the existing system, and

defining the comprehensive solution architecture built on four innovations: a 3D legal avatar, trilingual voice interaction, Tunisian legal intelligence, and real-time analytics.

- **Business Understanding:** Defines the strategic objectives of the project, identifies the needs of stakeholders including legal professionals and citizens, and specifies both functional and non-functional requirements to ensure legal accuracy, accessibility, and user experience.
- **Data Acquisition & Understanding:** Details the systematic collection and preprocessing of more than seventy official Tunisian legal documents. It explains their hierarchical structure, the article-level granularity, and the preprocessing techniques ensuring high-quality data suitable for AI-driven legal consultation.
- **Modeling:** Presents the architecture and workflow of the multi-agent RAG system designed for Tunisian law, covering hierarchical chunking, semantic embeddings, vector search, and comparative evaluation between Graph RAG and Vector RAG approaches, leading to the optimized system.
- **Deployment:** Focuses on transforming the developed models into a production-ready system through a React-based frontend, Flask backend, Firebase authentication, and Azure-based cloud deployment, ensuring scalability, security, and real-time accessibility.

Through this work, we aim to demonstrate how a comprehensive AI-driven approach can revolutionize legal service delivery, enhance accessibility for Tunisian citizens, and establish a scalable foundation for regional LegalTech innovation. The findings and implementations provide actionable insights for organizations seeking to leverage artificial intelligence for legal services transformation while maintaining the accuracy, security, and professional standards required in the legal sector

Chapter I: PROJECT GENERAL FRAMEWORK

1. Introduction

This chapter gives an overview of the context concerning the host organization where we executed our project, E-TAFAKNA. We will analyze the particular issues which motivated the initiative. Then, we will explain the original idea that we had. Lastly, we will give a summary of the detailed procedures that we followed in order to ensure the project development proceeded smoothly and was a success.

2. Host Organization Presentation

E-Tafakna is a pioneering Tunisian LegalTech startup founded in 2022, revolutionizing how businesses across the MENA region handle legal documents. The platform transforms contract management into a seamless digital experience, allowing users to create, collaborate, sign, and track legal documents from anywhere through an intuitive web and mobile interface.



Figure 1. 1: Logo E-tafakna

What sets E-Tafakna apart is its user-centric approach: over 100 ready-to-use contract templates (from NDAs to employment agreements), real-time tracking of document status, and multilingual support in Arabic, French, and English. By combining simplicity with powerful functionality, E-Tafakna makes professional legal services accessible and affordable for freelancers, entrepreneurs, and SMEs. Recognized with the StartUp'Act label and winner of the HiiL Innovating Justice Challenge 2024, the company is establishing itself as a regional leader in legal innovation.

3. General Project Overview

In this section, we will outline the general context of the project, beginning with an analysis of the existing system and its limitations, before presenting the proposed solution to address these challenges, and concluding with the methodological framework selection to guide the development process.

3.1 Project Framework

In the digital era, advanced AI platforms constitute strategic assets for organizations seeking to enhance accessibility, improve user interaction, and optimize service delivery.

Well-designed intelligent systems go beyond automated responses, becoming functional tools that centralize expertise, automate complex tasks, and provide seamless user experiences across multiple communication channels.

In the legal technology sector, this transformation is particularly crucial as traditional consultation methods create barriers for users seeking immediate, accessible legal guidance. Intelligent legal platforms must provide accurate information while adapting to diverse user needs, linguistic preferences, and varying technical proficiency levels, all while maintaining the trust and professionalism expected in legal services.

Within this perspective, E-Tafakna has undertaken a comprehensive modernization project of its legal AI capabilities, aiming to enhance user engagement, interaction quality, and service accessibility. The objective is to transform the current platform into a dynamic system integrating intuitive interfaces, multimodal communication features, and intelligent tools accessible across various devices. This project includes developing advanced knowledge management systems, incorporating legal document processing, intelligent retrieval mechanisms, conversation management with persistent memory, and real-time analytics capabilities. These developments aim to digitize legal consultation processes and improve user engagement and satisfaction.

3.2 Study of Existing Solution

E-Tafakna currently offers Elyssa AI, an intelligent legal assistant that represents a significant advancement in Tunisia's legal technology landscape. This AI system provides instant responses to legal questions through a conversational, mobile-friendly interface designed to make legal guidance more accessible to users. The platform operates as an interactive AI capable of responding to diverse legal inquiries, leveraging natural language processing to understand user questions and deliver contextually relevant answers. Elyssa AI's real-time consultation capability ensures immediate responses to legal questions, eliminating traditional delays associated with legal consultation processes. Through its conversational interface, the system enables natural language interaction, allowing users to communicate their legal concerns in everyday language rather than formal legal terminology. The platform's user-friendly design prioritizes accessibility through a mobile interface optimized for legal consultation, ensuring that legal assistance is available anytime and anywhere. This existing system has established E-Tafakna as a pioneer in Tunisian legal technology, providing a foundation for digital legal services while demonstrating the potential for AI-powered legal assistance in the MENA region.

3.3 Critical of the Existing System

Despite Elyssa AI's innovative approach to legal assistance in the Tunisian market, a thorough evaluation reveals several limitations that hinder optimal user experience. The platform operates within the constraints of traditional chatbot architecture, which limits the depth and richness of user interaction and focuses primarily on information delivery rather than creating an immersive consultation experience. From a technical perspective, the

system's knowledge base operates with general AI training that may not adequately capture the nuanced specificities of Tunisia's evolving legal landscape. Additionally, the current interface, though mobile-optimized, lacks the dynamic and engaging elements that modern users expect from advanced AI applications. The interaction flow remains linear and predictable, potentially reducing user engagement over time and limiting the platform's ability to accommodate diverse user preferences and communication styles essential for effective legal consultation services.

3.4 Problem Statement

With regards to the problem statement, we found three major issues with the functionalities of Elyssa AI:

Limited Legal Accuracy for Tunisian Context - Tunisian laws and procedures are not accurately incorporated into the legal framework.

Text-Only Communication Barriers - Users are unable to perform any actions except for typing and reading, creating accessibility hurdles.

Missing Voice Accessibility - Users are unable to interact with the device in a natural manner, eliminating hands free functionality.

These factors undoubtedly lower satisfaction levels and effectiveness for every Tunisia-based legal user, creating a strong gap for Tunisia to improve technologically to fill these needs and enhance the platform's strengths.

3.5 Proposed Solution

To address the identified limitations, we propose the development of a **Legal AI Avatar** an enhanced AI-powered legal assistant featuring four key technological innovations designed to revolutionize user interaction and service delivery.

3D Legal Avatar - A professional, lifelike avatar with synchronized speech and lip movements that creates a realistic, trust-building user experience, transforming impersonal text exchanges into engaging, human-like consultations.

Trilingual Voice Interaction - Natural speech communication capabilities in Arabic, French, and English, eliminating typing barriers and providing immediate verbal responses that accommodate Tunisia's linguistic diversity.

Tunisian Legal Intelligence - Specialized AI agents trained specifically on Tunisian legal frameworks, ensuring context-aware responses that accurately reflect local laws, procedures, and legal practices rather than generic legal guidance.

Real-Time Analytics - A comprehensive dashboard that tracks user engagement, consultation patterns, and system performance metrics to continuously monitor and improve service delivery effectiveness.

This integrated approach transforms the traditional legal consultation experience by combining cutting-edge AI technology with human-centered design principles, creating an accessible, engaging, and highly accurate legal assistance platform tailored to the specific needs of Tunisian users.

4. Adopted Methodology

4.1 Methodological Framework Selection

The selection of development methodology significantly influences AI project success rates. Current challenges in machine learning deployment demonstrate the critical importance of choosing appropriate frameworks for complex AI initiatives, particularly those involving breakthrough technologies and multimodal integration.

For Legal AI Avatar development, the challenge extends beyond traditional data science projects to encompass advanced AI integration, real-time voice processing, 3D avatar animation, and legal domain expertise—requiring a methodology capable of supporting breakthrough innovation while maintaining systematic rigor.

4.2 Comparative Analysis: CRISP-DM vs TDSP

Table 1. 1. Comparative Analysis of CRISP-DM and TDSP Methodologies

Technical Criteria	CRISP-DM (1999)	TDSP (2016)
Project Scope	Data mining and analytics	AI/ML intelligent applications
Development Approach	Sequential 6-phase process	Agile iterative methodology
Team Framework	Individual-focused work	Collaborative team structure with defined roles
Technology Integration	Limited to traditional data tools	Native integration with modern AI/ML platforms
Maintenance & Evolution	No active development	Continuously updated by Microsoft
Production Deployment	Analysis-oriented outcomes	Production-ready AI systems
Customer Validation	Implicit in evaluation phase	Dedicated customer acceptance stage
Flexibility	Linear progression	Cyclical with continuous feedback loops

4.3 TDSP Methodology Selection

TDSP offers a comprehensive approach specifically designed for artificial intelligence projects that aim to create intelligent applications. Unlike traditional data mining methodologies, TDSP addresses the complete lifecycle from conception to production deployment.

Selected for Technical Excellence:

AI-Native Design: Built specifically for intelligent applications using AI models

Production Focus: Addresses the critical gap where 87% of ML projects fail to reach production

Scalability Architecture: Supports enterprise-level AI deployment with robust infrastructure planning

Continuous Innovation: Cyclical process enabling continuous AI system improvement

TDSP Core Components

The TDSP methodology consists of four fundamental components that work synergistically to ensure project success:

1. Data Science Lifecycle Definition TDSP provides a well-defined iterative lifecycle that guides teams through systematic AI development phases. This lifecycle ensures comprehensive coverage of all project aspects from initial business understanding to continuous improvement, with clear deliverables and decision gates at each stage.

2. Standardized Project Structure A consistent, hierarchical project structure that promotes collaboration, reproducibility, and maintainability. This standardization includes predefined directories for data, code, documentation, and results, enabling seamless team collaboration and knowledge transfer across different project phases.

3. Recommended Infrastructure and Resources TDSP specifies the technical infrastructure requirements for successful AI project execution, including cloud computing resources, development environments, version control systems, and deployment platforms. This component ensures scalable and reliable infrastructure foundation for AI applications.

4. Recommended Tools and Utilities A comprehensive toolkit selection covering the entire AI development pipeline, including data processing frameworks, machine learning libraries, model deployment tools, and monitoring solutions. These tools are specifically chosen to support collaborative development and production-ready AI systems.

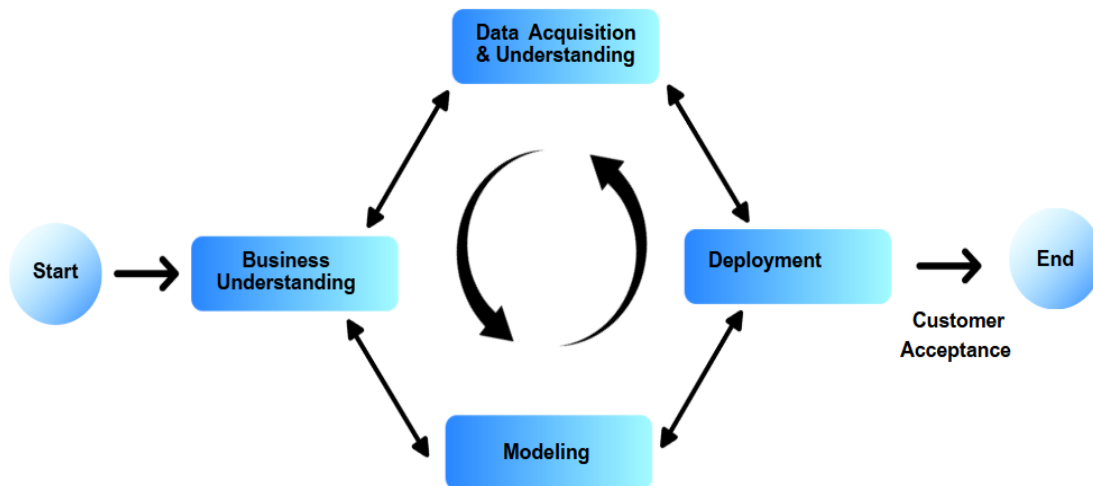


Figure 1. 2: TDSP Iterative Methodology

Adapted TDSP Lifecycle for Legal AI Avatar

Phase 1: Business Understanding & Legal Context Analysis

- Legal AI market analysis and user needs assessment
- E-Tafakna platform integration requirements definition
- Multimodal interaction objectives and success metrics establishment

Phase 2: Data Acquisition & Legal Knowledge Engineering

- Tunisian legal corpus collection and preprocessing
- Vector database architecture design for intelligent retrieval
- Multilingual dataset preparation (Arabic, French, English)

Phase 3: AI Model Development & Integration

- Legal reasoning AI agent development with specialized training
- Voice synthesis and recognition system implementation
- 3D avatar animation integration with synchronized speech

Phase 4: Production Deployment & Infrastructure

- React application development with secure authentication system (login/logout)
- Avatar interface integration with real-time user interaction capabilities
- Analytics dashboard for performance monitoring and user engagement tracking
- Scalable cloud deployment with real-time performance optimization

Phase 5: Customer Acceptance & Continuous Enhancement

- Legal expert validation and accuracy verification
- User acceptance testing with target demographics
- Performance monitoring and continuous AI model improvement

The TDSP methodology was selected as the optimal development framework for this AI-driven intelligent application.

5. Hierarchical Directory Structure

For the Legal AI Avatar project, we have implemented a streamlined three-folder structure that maintains TDSP principles while optimizing for our specific requirements. This structure represents a practical adaptation of the TDSP framework, focusing on essential components for AI-driven legal applications.

Our project follows a three-folder structure: **Docs** contains documentation and README files, **Data** houses clean legal documents and processed embeddings, and **Code** includes preprocessing scripts, React frontend, and Flask backend with Azure integrations.

E-TafaknaTNLegalAgent Project Structure:

```
project/
├── docs/
│   └── README.md
├── data/
│   ├── JSON_Files/
│   ├── new_file/
│   └── PDF_Code/
└── code/
    ├── __pycache__/
    ├── .github/
    ├── .vscode/
    ├── frontend/
    ├── static/
    ├── .env
    ├── .gitignore
    ├── l.json
    ├── app_manager.ps1
    ├── app_manager.sh
    ├── app.py
    ├── azureAiSpeechApp.yml
    ├── bash.exe.stackdump
    ├── database.py
    ├── Dockerfile
    ├── README.md
    ├── requirements.txt
    └── vad_iterator.py
```

5.1 Introduction to Practical Architecture

The Legal AI Avatar project implements a pragmatic three-folder architecture that balances TDSP standardization principles with development efficiency. This streamlined approach ensures rapid development cycles while maintaining the organizational benefits of the TDSP methodology. Each folder serves a distinct purpose in the AI system development lifecycle, from documentation and asset management to data processing and application deployment.

5.2 Documentation Directory (docs/)

The documentation directory serves as the central repository for project documentation and README files, maintaining a clean and focused approach to knowledge management.

README Documentation The docs directory primarily contains README files that provide:

- Project overview and objectives
- Installation and setup instructions
- Usage guidelines and examples
- Contribution guidelines for team members
- System requirements and dependencies

Documentation Philosophy This minimalist approach to documentation ensures that essential project information is easily accessible while avoiding documentation bloat. The focus on README files promotes clear, concise communication of project essentials without overwhelming team members with excessive documentation overhead.

Benefits of Streamlined Documentation

- **Quick Access:** Team members can rapidly find essential project information
- **Maintenance Efficiency:** Reduced documentation maintenance overhead
- **Focus on Essentials:** Emphasis on critical project information rather than exhaustive documentation
- **Developer-Friendly:** README-centric approach aligns with modern development practices

5.3 Data Directory (data/)

The data directory implements a clean, organized structure for managing the diverse datasets required for legal AI development.

JSON Files Directory (JSON Files/) Contains structured legal data in JSON format, including:

- Preprocessed Tunisian legal documents
- Legal case metadata and classifications
- User interaction patterns and preferences
- System configuration and parameter settings

New File Directory (new_file/) Serves as a staging area for:

- Newly acquired legal documents awaiting processing
- Updated legal texts and amendments
- User-contributed content and feedback
- Temporary data transformations during processing

PDF Code Directory (PDF_Code/) Houses legal documents in PDF format along with processing utilities:

- Original Tunisian legal PDF documents
- Document parsing and extraction scripts

Data Architecture Advantages This structure separates different data formats and processing stages, enabling efficient data pipeline management while maintaining clear data lineage and version control.

5.4 Data Directory (data/)

The code directory contains the complete application stack, integrating backend AI processing, frontend user interface, and deployment infrastructure.

Core Application Files

- **app.py** - Main Flask backend application with AI agent integration
- **database.py** - Database connectivity and data persistence layer

Frontend Development (frontend/) React-based user interface components including:

- 3D avatar rendering and animation systems
- Multilingual voice interaction interfaces
- User authentication and session management
- Real-time chat and legal consultation features

Configuration and Environment

- **.env** - Environment variables and sensitive configuration data

- requirements.txt - Python dependencies and version specifications
- .gitignore - Version control exclusion patterns
- Dockerfile - Container deployment configuration

Deployment and Management

- app_manager.ps1 & app_manager.sh - Cross-platform application management scripts
- azureAiSpeechApp.yml - Azure AI Speech Service configuration
- vad_iterator.py - Voice Activity Detection for speech processing

Development Tools

- .vscode/ - Visual Studio Code configuration and debugging settings
- .github/ - GitHub Actions CI/CD pipeline configuration
- __pycache__/ - Python bytecode cache for performance optimization

Static Assets (static/) Web assets including CSS stylesheets, JavaScript libraries, and multimedia resources for the user interface.

5.5 Integration Architecture Benefits

Streamlined Development Workflow The three-folder structure eliminates unnecessary complexity while maintaining essential organizational principles, enabling faster development cycles and reduced cognitive overhead for team members.

Scalable Foundation Despite its simplicity, this structure supports horizontal scaling through:

- Modular backend services in the code directory
- Flexible data processing pipelines in the data directory
- Comprehensive documentation management in the docs directory

Cross-Platform Compatibility The inclusion of both PowerShell (.ps1) and Bash (.sh) management scripts ensures seamless deployment across Windows and Unix-based systems, supporting diverse development environments.

6. Choice Of Services, Technologies, and Tools

6.1 Azure AI Search: Core Infrastructure Foundation

Azure AI Search is a scalable search infrastructure that indexes heterogeneous content and enables retrieval through APIs, applications, and AI agents. This platform serves as the foundational component of our legal AI system, providing advanced information retrieval capabilities specifically designed for modern AI applications.



Figure 1. 3: Azure AI Search Service

Core Capabilities:

The service handles both traditional search workloads and modern RAG (retrieval-augmented generation) patterns for conversational AI applications, making it ideally suited for our legal consultation system that requires both precise document retrieval and intelligent response generation.

6.2 Tunisia Legal Index Implementation

Our infrastructure implementation centers on a specialized Azure AI Search index named '**tunisialegalindex**' that serves as the central repository for processed Tunisian legal data.

Index Architecture:

Vector Search Capabilities: Inbound vectors are stored in vector indexes, enabling our legal AI system to perform semantic similarity searches across Tunisian legal documents. This capability allows the AI agents to identify relevant legal precedents and statutes based on contextual meaning rather than just keyword matching.

Data Processing Pipeline: The document format that Azure AI Search can index is JSON, requiring our legal documents to be preprocessed and structured appropriately. The system processes raw legal texts through:

- Document chunking and vectorization
- Multilingual content preparation
- Metadata extraction and classification
- Legal category and jurisdiction tagging

RAG Integration for Legal AI: The '**tunisialegalindex**' supports modern RAG (retrieval-augmented generation) patterns for conversational AI applications, enabling our AI agents to retrieve relevant legal information and generate contextually appropriate responses for user queries.

6.3 Resource Allocation and Management

6.3.1 Service Configuration

Search Service Tiers: The infrastructure utilizes appropriate Azure AI Search service tiers based on performance requirements and expected usage patterns for the legal AI avatar system.

Index Management: Systematic approach to index maintenance, updates, and optimization to ensure consistent performance as the legal document repository grows.

6.3.2 Integration Architecture

API Integration: Functionality is exposed through the Azure portal, simple REST APIs, or Azure SDKs, enabling seamless integration with the React frontend and Flask backend components of the legal AI avatar application.

Development and Testing: The Azure portal supports service administration and content management, with tools for prototyping and querying your indexes, facilitating efficient development and testing workflows.

6.4 Development Environment and Programming Languages

6.4.1 Visual Studio Code



Figure 1. 4: Visual Studio Code Logo

Definition and Purpose: Visual Studio Code is a lightweight but powerful source code editor which runs on your desktop and is available for Windows, macOS and Linux. It comes with built-in support for JavaScript, TypeScript and Node.js and has a rich ecosystem of extensions for other languages and runtimes.

Application in Legal AI Avatar: Visual Studio Code serves as the primary integrated development environment (IDE) for the Legal AI Avatar project, providing:

Unified development environment for both frontend (React) and backend (Flask) code

- Python extension support for AI/ML development
- Azure extensions for cloud service integration
- Git integration for collaborative development
- Debugging capabilities for complex AI workflows

6.4.2 Python



Figure 1. 5: Python Programming Language Logo

Definition and Purpose: Python is a high-level, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation. Python is dynamically typed and garbage-collected. It supports multiple programming paradigms, including structured, object-oriented and functional programming.

Application in Legal AI Avatar: Python serves as the primary backend programming language, enabling:

- Flask web framework implementation for API development
- AI/ML model integration and processing
- Azure services SDK integration
- Natural language processing for legal text analysis
- Database connectivity and data manipulation

6.4.3 JavaScript/HTML/CSS



Figure 1. 6: Web Technologies Stack Logo

Definition and Purpose: JavaScript is a programming language that is one of the core technologies of the World Wide Web, alongside HTML and CSS. HTML (HyperText Markup Language) is the standard markup language for documents designed to be displayed in a web browser. CSS (Cascading Style Sheets) is a style sheet language used for describing the presentation of a document written in HTML.

Application in Legal AI Avatar: These web technologies provide the foundation for frontend development:

- Interactive user interfaces for legal consultations
- Responsive web design for multi-device compatibility
- Real-time user interaction capabilities
- Integration with React framework components
- Voice interface controls and visual feedback systems

6.5 Frontend Framework

6.4.1 React

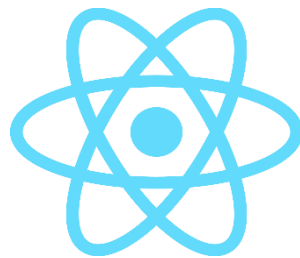


Figure 1. 7: React Framework Logo

Definition and Purpose: React is the library for web and native user interfaces. Build user interfaces out of individual pieces called components written in JavaScript. React is designed to let you seamlessly combine components written by independent people, teams, and organizations.

Application in Legal AI Avatar: React serves as the primary frontend framework for developing the user interface components of the Legal AI Avatar system. Its component-based architecture enables modular development of complex features including:

- 3D avatar rendering and animation controls
- Real-time chat interfaces for legal consultations
- Multilingual user interface components
- Voice interaction controls and feedback systems
- User authentication and session management

6.5 Backend Development Framework

Flask



Figure 1. 8: Flask Framework Logo

Definition and Purpose: Flask is a lightweight WSGI web application framework. It is designed to make getting started quick and easy, with the ability to scale up to complex applications. Flask is a micro web framework written in Python. It is classified as a microframework because it does not require particular tools or libraries.

Application in Legal AI Avatar: Flask provides the backend foundation for the Legal AI Avatar system, handling:

- RESTful API development for AI agent communication
- Database connectivity and data persistence
- Azure AI Search integration and query processing
- Voice processing and synthesis API endpoints
- User authentication and session management
- Legal document processing and analysis workflows

6.6 Azure Cloud Services Integration

6.6.1 Azure Cognitive Services Speech



Figure 1. 9: Azure Cognitive Services Speech Logo

Definition and Purpose: Azure Cognitive Services Speech provides speech-to-text, text-to-speech, speech translation, and speaker recognition capabilities through REST APIs and Speech SDK. The service supports real-time or batch transcription with accurate results.

Application in Legal AI Avatar: Azure Speech Services enable comprehensive voice functionality including:

- Multilingual speech recognition (Arabic, French, English)
- Text-to-speech synthesis for avatar communication
- Real-time voice processing for legal consultations
- Voice activity detection and audio preprocessing
- Integration with legal AI responses for natural conversations

6.6.2 Azure OpenAI



Figure 1. 10: Azure OpenAI Service Logo

Definition and Purpose: Azure OpenAI Service provides REST API access to OpenAI's powerful language models including GPT-4, GPT-3.5-Turbo, and Embeddings model series. These models can be easily adapted to your specific task including but not limited to content generation, summarization, semantic search, and natural language to code translation.

Application in Legal AI Avatar: Azure OpenAI powers the legal intelligence capabilities through:

- Legal reasoning and case analysis
- Multilingual legal document interpretation
- Contextual legal advice generation
- Legal precedent analysis and recommendations
- Integration with Tunisian legal corpus for accurate responses

6.6.3 Azure AI Foundry



Figure 1. 11: Azure AI Foundry Logo

Definition and Purpose: Azure AI Foundry is a unified platform for building, deploying, and managing AI applications. It provides tools for model training, fine-tuning, and deployment with comprehensive monitoring and governance capabilities.

Application in Legal AI Avatar: Azure AI Foundry manages the AI agent infrastructure through:

- Legal AI agent development and deployment
- Model fine-tuning for Tunisian legal context
- Performance monitoring and optimization
- AI model governance and compliance
- Integration orchestration between multiple AI services

6.6.4 Azure Identity



Figure 1. 12: Azure Identity Logo

Definition and Purpose: Azure Identity provides identity and access management capabilities, enabling secure authentication and authorization for applications and users. It supports modern authentication protocols and provides comprehensive security features.

Application in Legal AI Avatar: Azure Identity ensures secure access management through:

- User authentication and authorization
- Role-based access control for legal professionals
- Secure API access management
- Integration with organizational identity systems
- Compliance with legal industry security standards

6.7 Authentication and Database Services

Firestore



Figure 1. 13: Firestore Logo

Definition and Purpose: Firestore is a platform developed by Google for creating mobile and web applications. It provides a realtime database, user authentication, hosting, and other services that help developers build high-quality apps quickly.

Application in Legal AI Avatar: Firestore provides comprehensive backend services including:

User authentication and session management

- Real-time chat history storage and synchronization
- Secure user profile and preference management
- Real-time database for legal consultation records
- Cross-platform compatibility for web and mobile access

Key Features for Legal AI System:

- **Authentication Services:** Secure user login and registration
- **Realtime Database:** Chat history and conversation persistence
- **Cloud Functions:** Serverless backend logic execution
- **Security Rules:** Document-level access control for legal data

6.8 Technology Stack Architecture

6.8.1 Integrated Development Workflow

The standardized tech stack creates a cohesive development environment where:

- **Frontend Layer:** React/JavaScript/HTML/CSS provides responsive user interfaces with real-time interaction capabilities, enabling seamless integration with voice and avatar features.
- **Backend Layer:** Python/Flask offers lightweight and scalable API development, facilitating integration with Azure AI services and database management.

- **AI Services Layer:** Azure OpenAI, Azure Speech, and Azure AI Foundry work together to provide comprehensive legal AI capabilities including natural language processing, voice interaction, and intelligent agent management.
- **Security and Authentication:** Azure Identity and Firebase ensure robust security measures while providing user-friendly authentication experiences.

6.8.2 Development Environment Integration

Visual Studio Code serves as the central hub connecting all technologies through:

- Python extensions for backend development
- JavaScript/React extensions for frontend development
- Azure extensions for cloud service integration
- Git integration for version control and collaboration

7. Conclusion

To conclude, this chapter provided an overview of E-Tafakna as the host organization, highlighting its pioneering role in the LegalTech sector and the current state of its Elyssa AI platform. We examined both the strengths and the shortcomings of the existing system, particularly the lack of contextual accuracy, limited interaction modes, and absence of voice accessibility. These limitations led us to define the problem statement and propose the Legal AI Avatar as an innovative solution that combines multilingual voice interaction, Tunisian legal intelligence, and real-time analytics. By establishing this foundation, we not only clarified the motivations behind the project but also set the methodological and technical direction for its development.

The following chapter, **Business Understanding**, will build on these insights by analyzing the business objectives, constraints, and user requirements that guided the design of our proposed solution.

Chapter II: BUSINESS UNDERSTANDING

1. Introduction

The Business Understanding phase represents the foundation of the Team Data Science Process (TDSP) methodology. Its primary purpose is to align the project objectives with the actual business needs of the organization, ensuring that the proposed technical solutions directly address real-world challenges. In the context of this study, the Business Understanding phase focuses on identifying the legal information accessibility gap in Tunisia, analyzing the requirements of key stakeholders, and defining the objectives and success criteria of the Legal AI Avatar project. This step provides a clear bridge between the business context and the data science approach, guaranteeing that subsequent phases data preparation, modeling, and deployment are fully aligned with the strategic vision of the company E-Tafakna.

2. Business Understanding

2.1 Business Objectives

2.1.1 Primary Goal :

Develop an AI-powered avatar capable of answering legal questions related to Tunisian law, making legal information accessible to citizens and legal professionals.

2.1.2 Strategic Objectives:

The Legal AI Avatar project is structured around four strategic pillars that address both market opportunities and technological innovation:

Innovation Leadership Our primary objective centers on establishing Tunisia's first legal AI avatar featuring comprehensive multi-modal interaction capabilities. This innovation positions E-Tafakna as a technology pioneer in the MENA legal technology sector, creating sustainable competitive advantages through breakthrough AI implementation.

Legal Accuracy Excellence We aim to achieve expert-validated legal responses that meet professional standards required by Tunisia's legal community. This objective ensures our system delivers reliable, accurate information validated by qualified legal professionals, establishing credibility and trust within the legal ecosystem.

Universal Accessibility The project prioritizes inclusive design through trilingual support encompassing Arabic, French, and English languages. This accessibility framework eliminates linguistic barriers, enabling comprehensive legal information access for Tunisia's diverse population and international business community.

Superior User Experience Our objective focuses on developing an intuitive avatar interface integrated with natural voice interaction capabilities. This approach transforms

traditional legal consultation experiences by providing engaging, human-like interactions that simplify complex legal information access.

2.2 Project Team Structure

Table 2. 1: Project Team Roles and Responsibilities

Role	Key Responsibilities
Product Owner: Lawyer/Legal Expert	• Evaluate assistant outputs against Tunisian law standards. • Identify corrections and improvements needed. • Provide detailed feedback on legal compliance issues. • Approve final system responses for deployment. • Provide missing laws and new Tunisian legal updates.
Scrum Master: Expert Data Scientist	• Collect and analyze Product Owner feedback and remarks. • Translate business requirements into technical specifications. • Communicate correction requirements to Development Team. • Contribute to system development and optimization • Coordinate between Product Owner and Development Team
Development Team: Data Scientist	• Implement requirements and corrections from Scrum Master • Apply feedback during development process • Modify AI models based on legal accuracy requirements. • Execute technical improvements for system optimization.

2.3 End User Communities

Legal Professionals

Lawyers, legal researchers, paralegals, and legal consultants who require reliable, accurate legal information for professional practice. Their needs include rapid access to current legislation, case precedents, and regulatory updates.

General Public

Citizens, entrepreneurs, and small business owners seeking accessible legal guidance and information. This stakeholder group prioritizes user-friendly interfaces, multilingual support, and simplified legal explanations.

2.4 Comprehensive Business Requirements

2.4.1 Functional Requirements Analysis

Legal Question Processing System The system must demonstrate capability to understand, analyze, and respond to complex legal inquiries related to Tunisian law. This

includes natural language processing for legal terminology, context understanding, and generation of accurate, relevant responses based on current legislation and legal precedents.

Source Citation & Referencing Framework Implementation of automated citation systems that provide precise references to relevant legal articles, laws, regulations, and court decisions. This functionality ensures transparency, enables verification, and maintains professional standards expected in legal information systems.

Multilingual Communication Infrastructure Comprehensive language support enabling query processing and response generation in Arabic, French, and English. This includes language detection, translation capabilities, and culturally appropriate legal terminology usage across all supported languages.

Multimodal Interaction Platform Integration of text-based communication, voice interaction, and 3D avatar visual representation. This multimodal approach accommodates diverse user preferences, accessibility needs, and interaction styles while maintaining consistent functionality across all modes.

User Management & Security System Secure authentication mechanisms, personalized user profiles, conversation history management, and privacy protection features. This system ensures data security while providing personalized experiences and maintaining legal confidentiality standards.

Legal Document Search & Retrieval Advanced search functionality enabling users to locate, access, and retrieve specific legal documents, regulations, and legislative texts. This includes intelligent indexing, content categorization, and relevance ranking based on user queries.

2.4.2 Non-Functional Requirements Framework

Legal Accuracy & Compliance Standards All system responses must undergo validation by qualified legal experts to ensure 100% compliance with Tunisian law. This includes regular accuracy audits, legal review processes, and continuous validation protocols to maintain professional standards.

Performance & Responsiveness Criteria The system must deliver real-time responses with sub-second latency to maintain smooth, interactive user experiences. Scalable architecture, and consistent response times across all interaction modes.

Data Security & Privacy Protection Comprehensive data protection protocols ensuring user privacy, confidential information security, and compliance with international privacy standards. This includes encryption, secure data storage, audit trails, and privacy compliance monitoring.

User Interface & Accessibility Standards Intuitive interface design supporting users with varying technical proficiency levels. Accessibility features must comply with international standards, supporting users with disabilities and ensuring inclusive design principles across all interaction modes.

Content Maintainability & Update Mechanisms Architecture enabling dynamic legal content updates without system downtime or service interruption. This includes automated content management, version control for legal documents, and seamless integration of legislative changes and updates.

3. Solution Approach Overview

Our solution architecture contains a 12-step process designed to transform traditional legal consultation into an intelligent, accessible service. The approach systematically processes Tunisian law documents through cleaning, structuring, and intelligent storage, enabling rapid response to user queries through AI-powered interaction.

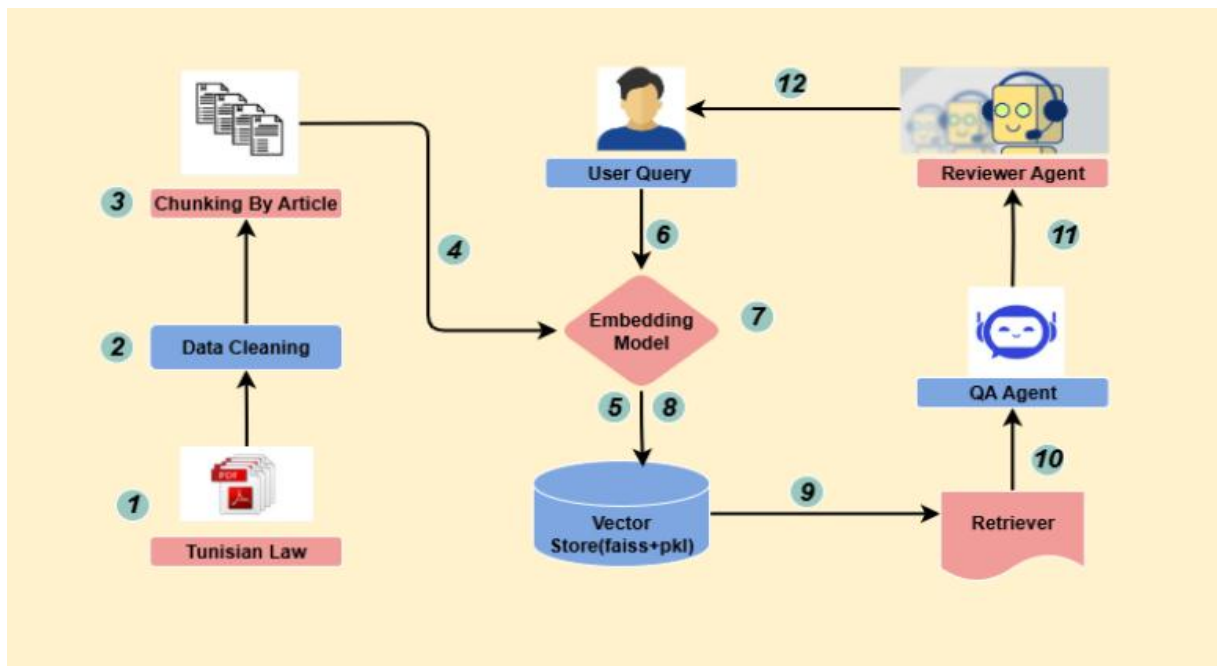


Figure 2. 1: Legal AI Avatar Solution Architecture (Business Process Flow)

3.1 Business Process Steps

Step 1: Legal Document Collection Systematic gathering of official Tunisian legal documents from authoritative governmental sources to build comprehensive legal knowledge base.

Step 2: Content Standardization Professional cleaning and formatting of legal documents to ensure consistency and remove non-essential elements that could affect system accuracy.

Step 3: Legal Structure Preservation Intelligent segmentation of legal content by individual articles of Tunisian legal framework.

Step4: Semantic Encoding Transforming segmented legal content into numerical embeddings that capture semantic meaning for AI analysis, ensuring preservation of legal context and interpretation.

Step 5: Knowledge Base Creation Systematic storage of processed legal information with metadata saved in .pkl files and vector embeddings stored in .faiss files, creating an intelligent repository that enables rapid content retrieval and semantic understanding.

Step 6: User Query Reception Capturing user legal questions through text input across supported languages.

Step 7: Query Intelligence Converting user questions into vector embeddings to enable rapid similarity search within the FAISS vector database for retrieving the most relevant legal information.

Step 8: Relevance Matching Performing L2 distance-based similarity search using FAISS IndexFlatL2 with AVX2 optimization to identify and rank the most pertinent legal provisions and articles addressing the user's specific inquiry

Step 9: Content Retrieval Extracting relevant legal documents and supporting information using LangChain retriever to fetch the most pertinent legal provisions, citations, and documentation based on the similarity search results

Step 10: Response Formulation Sending the extracted relevant documents to the Question Answering agent for processing and generating comprehensive, understandable legal responses that combine retrieved legal content with contextual explanations.

Step 11: Response Enhancement Forwarding the QA agent's output to a validation system for accuracy improvement, legal compliance verification, and professional standard alignment.

Step 12: User Response Delivery Sending the final validated response to the user.

3.2 Technical Terms Explained

FAISS (Facebook AI Similarity Search)

- Library developed by Meta for large-scale vector similarity search
- Enables fast searches through millions of vectors
- IndexFlatL2: uses Euclidean distance (L2) to measure similarity
- between vectors

PKL (Pickle Files)

- Python serialization format that saves Python objects into binary files
- Preserves metadata, data structures, and textual information
- Enables rapid loading of complex data

AVX2 (Advanced Vector Extensions 2)

- Intel/AMD processor instructions for accelerating vector calculations
- Optimizes mathematical operations on embeddings
- Significantly improves FAISS search performance

Embeddings/Vector Embeddings

- Numerical representations of text as multidimensional vectors
- Capture semantic meaning and context of legal content
- Enable mathematical comparison between texts

L2 Distance (Euclidean Distance)

- Method for calculating similarity between two vectors
- Lower distance indicates higher semantic similarity between documents

LangChain Retriever

- Framework that interfaces vector search systems with language models
- Manages extraction and formatting of relevant documents

4. Conclusion

In conclusion, the Business Understanding phase has defined the project's objectives, stakeholders, and requirements, ensuring alignment between business needs and the data science approach. These foundations provide a clear direction for the next phase of the TDSP lifecycle: Data Acquisition and Understanding.

Chapter III: Data Acquisition & Understanding

1. Introduction

The success of any data-driven legal information system relies heavily on the quality and suitability of the underlying dataset. In this chapter, we present the process of acquiring Tunisian legal documents and provide a detailed understanding of their structure and content. This step ensures that the collected corpus is both representative of the Tunisian legal framework and appropriate for building a reliable legal question-answering system.

2. Data Acquisition & Understanding

2.1 Data Acquisition

We collected more than 72 official Tunisian legal documents (PDF format) from governmental sources. These documents include **laws, legal codes, and regulations** that represent the fundamental pillars of Tunisia's legal framework. All documents are written in **French**, the official legal language of the Tunisian judicial system.

Each document was subjected to systematic **cleaning and preprocessing**, which involved:

- Removing formatting inconsistencies (headers, footers, page numbers, line breaks)
- Extracting pure textual content
- Preserving legal terminology and citation accuracy to maintain the integrity of the legal meaning.

2.2 Data Understanding:

2.2.1 Structural Analysis

Analysis revealed that Tunisian legal documents follow a **strict hierarchical structure** inspired by the French legal system.

The structure can be represented as:

Law → Title → Chapter → Section → Article

- **Law:** The name of the legal text (e.g., “Code du Travail”)
- **Title:** First level inside the law, grouping related legal themes
- **Chapter:** Second level, grouping detailed topics inside each title
- **Section:** Third level, refining content within a chapter
- **Article:** Final level, containing the actual legal provisions

Each law contains **multiple titles**, each title may have **zero or more chapters**, each chapter may have **zero or more sections**, and each section contains **zero or more articles**. This hierarchy allows **precise navigation from general principles down to specific provisions**.

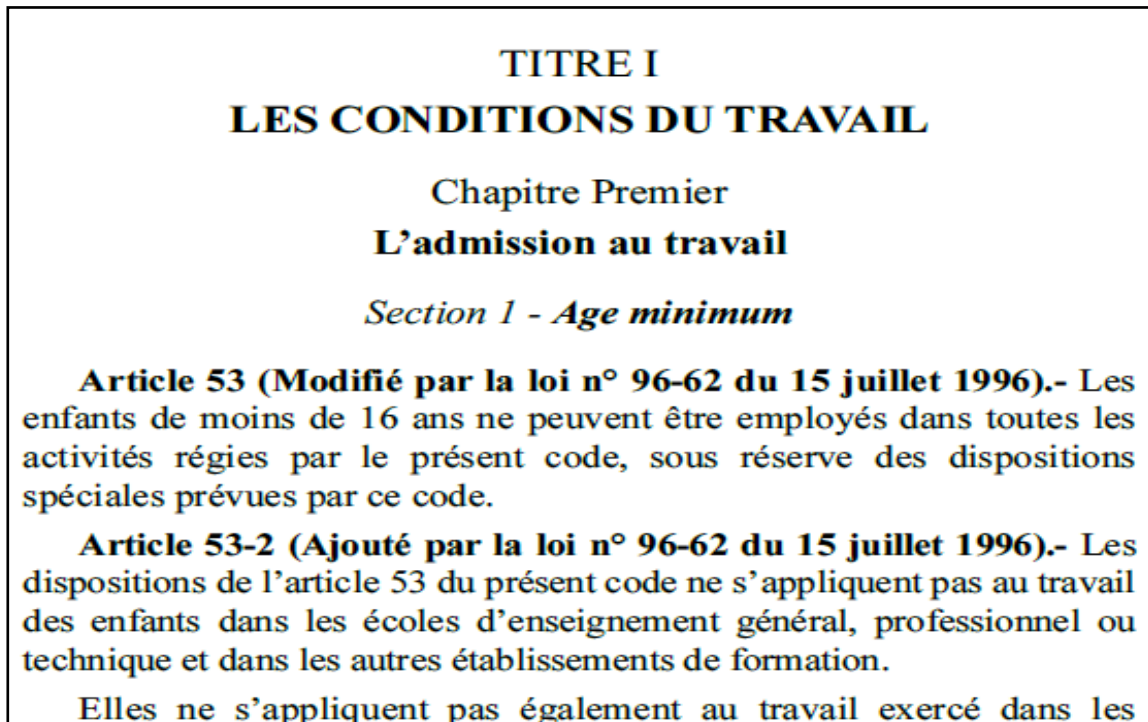


Figure 3. 1: Example of hierarchical structure in Tunisian legal documents

The figure above (see **Figure 1**) shows a real example from the *Code du Travail* illustrating the hierarchical organization:

- **Title I – Conditions de Travail**
- **Chapter Premier – L’admission au travail**
- **Section 1 – Age minimum**
- **Article 53 and Article 53-2 with their legal provisions**

This structure confirms the consistent organization of the legal texts.

2.2.2 Exploratory Data Analysis (EDA)

A brief EDA was conducted to understand the composition and quality of the dataset:

- **Volume and coverage:**
 - ❖ 72 legal documents collected
 - ❖ 18,349 individual articles extracted
- **Data quality:**
 - ❖ No missing article texts

- ❖ Legal terminology preserved accurately
- **Distribution insights:**
 - ❖ Most laws contain between 3 and 8 titles
 - ❖ Average of 5 chapters per title
 - ❖ Article counts vary widely (from 10 to 300+ per law)
- **Content readability:**
 - ❖ Clean and consistent text, enabling future NLP tasks

This EDA confirmed that the dataset is complete, clean, and well-structured for use in building a legal question-answering system.

2.2.3 System Suitability

Based on the analysis, the dataset is appropriate for legal Q&A system development due to:

- Article-level granularity enabling precise question answering
- Hierarchical structure supporting contextual legal understanding
- Comprehensive coverage across major Tunisian legal domains
- High content quality ensuring reliable information delivery

3. Conclusion

The acquisition and analysis of Tunisian legal documents confirmed the availability of a high-quality, well-structured, and representative dataset. This dataset forms a solid foundation for the development of a legal Q&A system, ensuring both accuracy and reliability in addressing legal queries. Building on this dataset, the next step involves the modeling phase, where appropriate algorithms and techniques will be applied to extract meaningful insights, enable precise question-answering, and optimize the system's performance.

Chapter IV: MODELING

1. Introduction

Chapter 5 focuses on the design, implementation, and evaluation of the legal AI system's modeling component. It presents the architecture and workflow of a multi-agent RAG system tailored for Tunisian legal documents, highlighting hierarchical chunking, semantic embedding, and vector search strategies. The chapter also examines the initial system limitations, iterative improvements, and the comparative evaluation between Graph RAG and Vector RAG approaches. By the end of this chapter, the foundation for an efficient, accurate, and context-aware legal AI system is established, paving the way for deployment and real-world application.

2. Modeling

2.1 Initial RAG Implementation

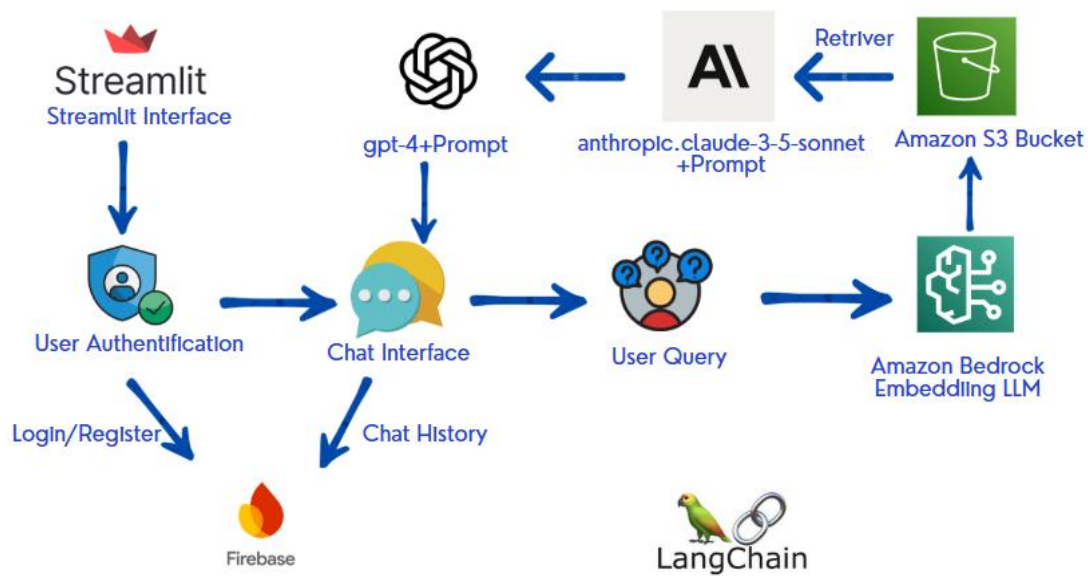


Figure 4. 1: Multi-Agent Legal RAG Architecture

As illustrated in Figure 2.2 The technical architecture implements a sophisticated multi-agent RAG system where users authenticate through Firebase and interact via Streamlit interface to submit legal queries. The system converts user queries into vector embeddings using Amazon Bedrock LLM, then performs L2 distance-based similarity search through FAISS IndexFlatL2 with AVX2 optimization to identify relevant legal documents from Amazon S3 storage. Retrieved documents are processed by a GPT-4o QA Agent with specialized legal prompts to generate comprehensive responses, followed by validation through a reviewer agent for accuracy verification. The entire workflow is orchestrated via LangChain framework, with user data and chat history managed through Firebase Firestore,

enabling secure, real-time legal consultation with persistent conversation tracking and multi-stage quality assurance.

2.2 Evaluation of Initial Implementation

2.2.1 Performance Analysis & Critical Issues

Legal Accuracy Problems:

- **Article Fragmentation:** Legal provisions spanning multiple articles were treated as isolated rules
- **Hierarchy Misunderstanding:** Constitutional principles weren't prioritized over secondary regulations

Context Loss Issues:

- **Cross-Referential Breakdown:** Legal articles containing references to other provisions were processed in isolation
- **Semantic Relationship Loss:** System failed to maintain contextual bridges between interconnected legal sections
- **Structural Fragmentation:** Article-based chunking severed critical relationships between related legal provisions

Incorrect Response Example: When asked: "My father passed away and he has a wife who has no children from him. Do I have to give her part of my dad's land?"

System's Wrong Response: "According to Article 101, the wife inherits $\frac{1}{4}$ or $\frac{1}{2}$ of the estate."

Critical Error: The system referenced Article 101 which applies to different inheritance scenarios. The correct answer should reference Article 102: the wife inherits $\frac{1}{4}$ if the deceased has no children and $\frac{1}{8}$ if he has children. This demonstrates the system's failure to apply conditional legal logic based on specific family circumstances.

2.2.2 Team Project Meeting & Strategy Revision

Following evaluation results, the project team convened to address critical system limitations and develop an enhanced implementation strategy requiring fundamental architectural changes.

❖ Chunking Strategy Enhancement:

- Implement hierarchical chunking preserving legal document structure and cross-references

- Maintain contextual relationships between interconnected legal provisions

❖ Platform Migration:

- Migrate to Azure ecosystem for improved AI capabilities and legal document processing
- Implement multi-agent validation with legal domain specialization

❖ RAG Architecture Evolution:

- Comparative evaluation between current and enhanced RAG implementations needed
- Establish legal professional feedback integration for continuous improvement

2.3 Solution Architecture Overview

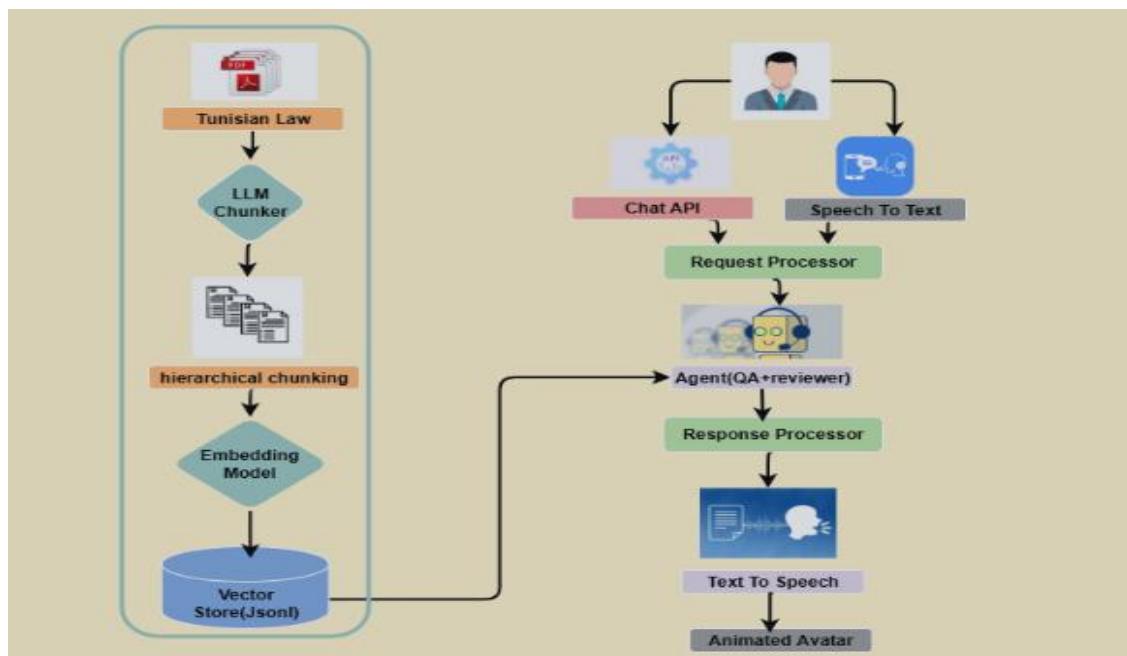


Figure 4. 2: Multi-Agent Legal RAG Architecture

As illustrated in Figure 4.2, Our Legal AI Avatar system transforms Tunisian legal documentation through a comprehensive business process that begins with official legal PDFs collected from governmental sources, which undergo intelligent content structuring by organizing information according to Tunisia's legal hierarchy (law, chapter, article), followed by metadata extraction and semantic indexing to capture essential legal information including document sources, publication dates, and legal references. Each structured piece of legal content undergoes semantic embedding generation and systematic organization in JSON format for efficient storage and retrieval, while Azure AI Search indexing creates a searchable knowledge base supporting both semantic understanding and vector-based content matching. Users engage with the system through flexible input methods including text-based queries and natural voice communication, triggering an intelligent legal agent to perform comprehensive searches across the indexed legal knowledge base to identify the most relevant legal provisions and supporting documentation. The system generates comprehensive legal

responses combining retrieved legal content with contextual explanations and appropriate citations to Tunisian legal sources, delivering final responses through an animated avatar interface that provides engaging, human-like interaction while presenting complex legal information in an accessible format, ensuring that citizens, legal professionals, and students receive accurate, accessible, and professionally validated legal guidance while maintaining the integrity and authority of Tunisian legal information.

2.4 Hierarchical Chunking & Embedding Implementation

Our system implements intelligent hierarchical chunking designed for Tunisian legal document architecture following the French legal structure: Law → Title → Chapter → Section → Article. The extraction process leverages GPT-4o's capabilities with engineered prompts that recognize legal boundaries and preserve semantic integrity through context-aware chunking at article-level divisions.

2.4.1 Advanced Embedding Generation ProcessFollowing hierarchical extraction,

The system generates 1536-dimensional vector representations using Azure OpenAI's advanced embedding models. Each vector captures semantic meaning, enabling understanding of conceptual relationships between legal provisions beyond simple keyword matching. Each legal article receives comprehensive metadata including unique identification, hierarchical positioning, and semantic vector representations.

2.4.2 Vector Search Infrastructure: "tunisialegalindex"

The screenshot displays the Microsoft Azure AI Search portal for the 'tunisialegalindex'. The interface includes a sidebar with navigation options like 'Accueil', 'Créer', 'Tous les services', 'Tableau de bord', 'Toutes les ressources', 'Groupes de ressources', 'App Services', 'Application de fonction', 'Bases de données SQL', 'Azure Cosmos DB', and 'Azure AI Foundry'. The main content area shows the index configuration with various tabs and a table of fields.

Nom du champ	Type	Récupérable	Filtrable	Triable	À choix mi	Peut faire l'obj	Analyseur
id	String	☒	☒	☒	☐	☐	
loi	String	☒	☒	☒	☒	☒	Standa... ▼
titre	String	☒	☐	☐	☐	☒	Standa... ▼
chapitre	String	☒	☐	☐	☐	☒	Standa... ▼
section	String	☒	☐	☐	☐	☒	Standa... ▼
article	String	☒	☐	☐	☐	☒	Standa... ▼
vector	SingleCollection	☐				☒	

Figure 4. 3: Azure AI Search Index Configuration for Tunisian Legal Documents

As illustrated in Figure 4.3, Our system creates the "tunisialegalindex" within Azure AI Search, implementing advanced search capabilities combining traditional text search with vector similarity matching.

HNSW Algorithm Implementation: The system utilizes Hierarchical Navigable Small World (HNSW) algorithm configured as "myHnsw" for efficient vector similarity search. HNSW constructs multi-layer graph structures where each layer contains nodes (legal documents) connected by similarity relationships, achieving logarithmic search complexity while maintaining high accuracy.

➤ **Search Index Field Architecture:**

- id: Unique document key for efficient indexing and filtering
- Law, Title, Chapter, Section: Hierarchical legal fields enabling precise navigation and filtering
- Article: Complete article content with full-text search capabilities
- vector: 1536-dimensional semantic embeddings for advanced similarity matching

➤ **Azure AI Search Query Processing:**

The "tunisialegalindex" implements hybrid search capabilities combining keyword-based searches with vector similarity matching. When users submit queries, the system simultaneously performs full-text search across hierarchical fields and semantic vector search using query embeddings matched against stored legal document vectors.

➤ **Query Processing Workflow:**

Query Analysis: User queries undergo embedding transformation

- Parallel Search Execution: Simultaneous text-based and vector-based searches
- HNSW Vector Navigation: Rapid navigation through graph layers for similar content
- Results Fusion: Combination of text search and vector similarity matches
- Relevance Ranking: Final scoring considering both textual and semantic similarity

This approach ensures the "tunisialegalindex" delivers precise, contextually relevant legal information through AI-powered search capabilities while maintaining Tunisian legal documentation integrity.

2.5 RAG Architecture Comparative Study

Before implementing our legal assistant's RAG solution, we conducted a systematic comparison between Graph RAG and Vector RAG approaches to determine the optimal architecture for processing Tunisian legal documents.

2.5.1 Advanced Embedding Generation Process

Our Graph RAG implementation began by transforming structured JSON legal documents containing hierarchical levels (Law, Title, Chapter, Section, Article) into a Neo4j graph database. Using the automated transformation pipeline, we created nodes for each legal element and established "CONTIENT" relationships between hierarchical levels, preserving the complete legal document structure.

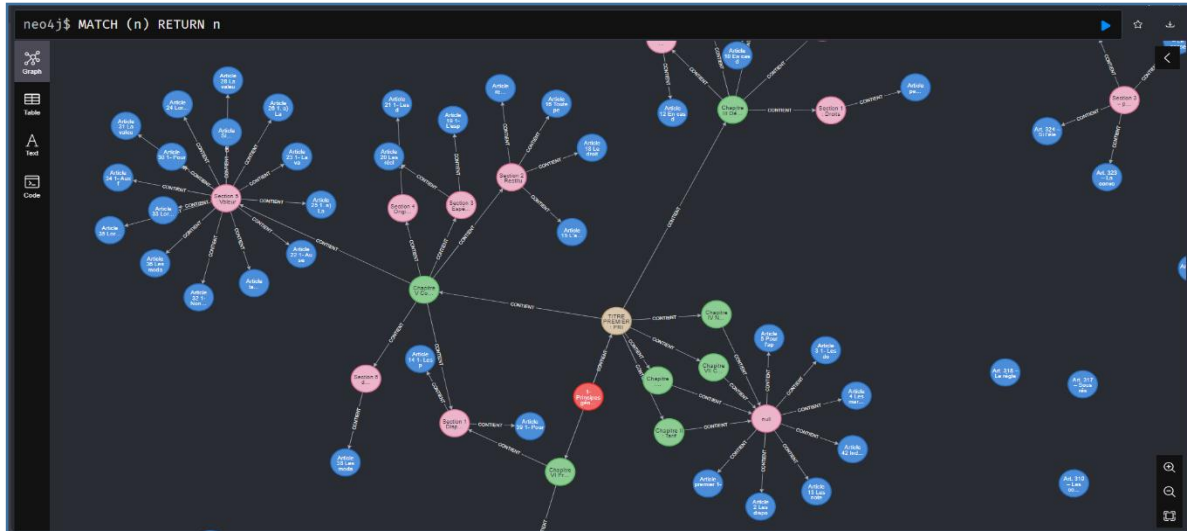


Figure 4. 4: Neo4j Browser Graph Visualization

We developed a Streamlit interface that extracts legal keywords from user queries, generates dynamic Cypher queries for graph traversal, and integrates with GPT-4o via Azure OpenAI for natural language response generation. The system demonstrated comprehensive context preservation and transparent traceability of legal relationships, allowing users to visualize document connections through NetworkX graphs.



Figure 4. 5: Neo4j Browser Graph Visualization

However, the complex graph traversal operations resulted in average response times of 8-12 seconds, and the infrastructure required specialized Neo4j expertise with higher operational complexity.

2.5.2 Vector RAG Implementation

The Vector RAG approach leveraged Microsoft Azure AI Foundry's multi-agent architecture, featuring two specialized agents: a Question Answering Agent that utilizes the Azure AI Search index (tunisialegalindex) for semantic similarity matching, and a Reviewer Agent that validates and enhances response accuracy for legal coherence. The system processes user queries through vectorization, performs similarity search across the legal document index, and generates responses via the collaborative agent workflow. This approach achieved superior performance with average response times of 6 seconds. The architecture benefits from automated maintenance, native cloud scaling capabilities, and operational simplicity, requiring minimal specialized expertise for deployment and management.

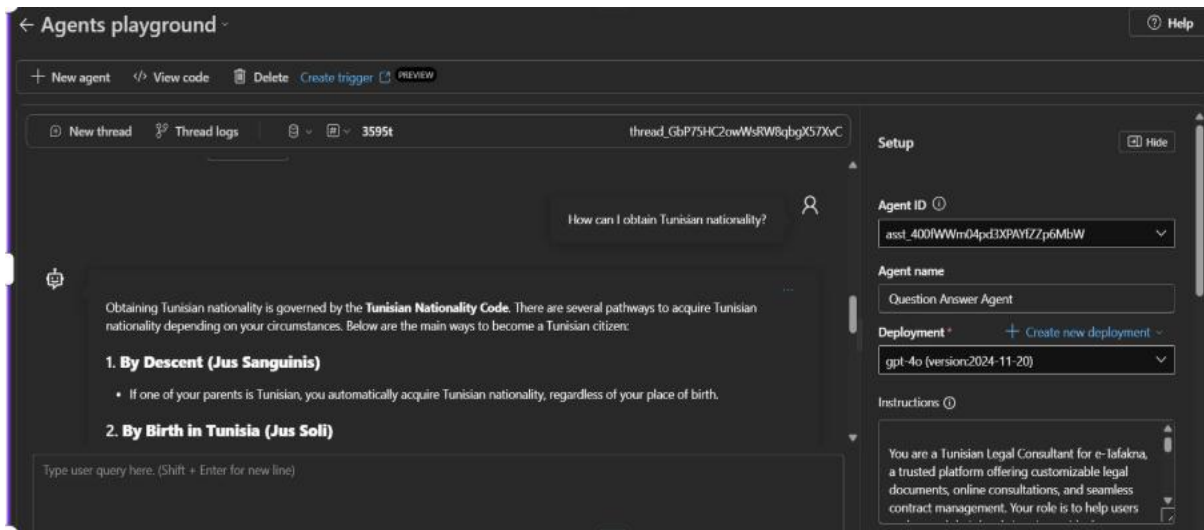


Figure 4. 6 : Azure AI Foundry Multi-Agent Configuration

2.5.3 Comparative Analysis and Decision

Our systematic evaluation revealed that Vector RAG outperformed Graph RAG in key metrics: response times were 50% faster (6 seconds vs 8-12 seconds), while response quality remained. The economic analysis showed significant cost advantages for Vector RAG, with Azure AI Search costing \$242/month compared to Neo4j AuraDB's \$525.60/month, resulting in monthly savings of \$283.60 (54% cost reduction). Although Graph RAG provided superior hierarchical context preservation and relationship visualization capabilities, Vector RAG's performance efficiency, economic viability, and operational simplicity made it the optimal choice. We selected Vector RAG with hierarchical metadata enrichment as our final architecture, implementing a hybrid approach that preserves legal document structure through enhanced indexing while maintaining the performance and cost advantages of vector-based retrieval.

2.6 System Design and Technical Implementation Process

This section presents a detailed, end-to-end breakdown of the real-time AI voice agent system, highlighting the technical design choices, processing stages, and cloud-native mechanisms involved. The architecture adopts modularity, scalability, and service abstraction to deliver seamless multimodal interactions through an intelligent web-based assistant.

2.6.1 System Overview and Process Foundation

The application architecture is structured around three tightly integrated architectural layers that work in harmony to deliver production-grade conversational AI capabilities:

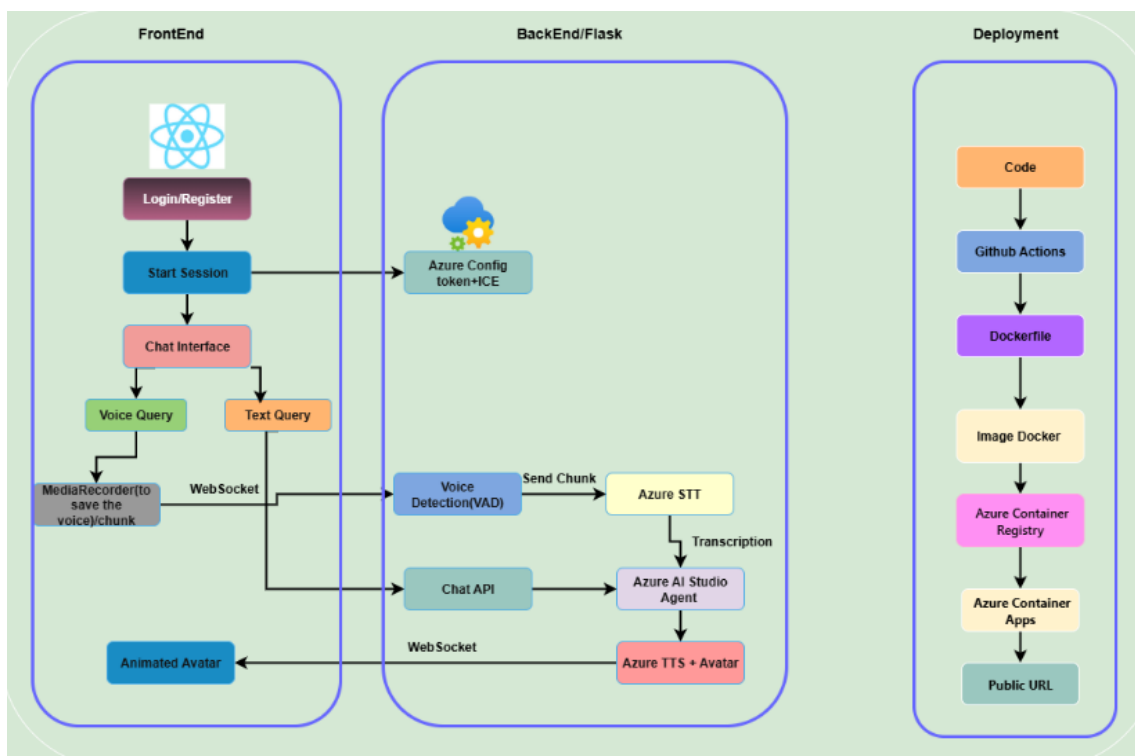


Figure 4. 7: Complete Application Process Flow - Real-Time AI Voice Agent System

Frontend Layer: A React-based web application that serves as the primary user interface, incorporating advanced audio capture through the MediaRecorder API, real-time WebSocket communication via Socket.IO, and immersive WebRTC streaming for synchronized avatar visualization with lip-sync accuracy.

Backend Layer: A robust Python Flask server enhanced with Flask-SocketIO capabilities, orchestrating the complete conversational pipeline including Voice Activity Detection (VAD), speech recognition, natural language processing through OpenAI integration, text-to-speech synthesis, and seamless communication with Azure cloud services.

Cloud Infrastructure Layer: A comprehensive Azure-based ecosystem leveraging Speech-to-Text (STT), Text-to-Speech (TTS), Azure AI Studio Agent services, Firebase Authentication and Firestore for data persistence, all deployed through Azure Container Apps for elastic scalability and high availability.

2.6.2 User Authentication and Session Management Process

❖ Step 1: Secure User Authentication and Identity Management

User access is managed through Firebase Authentication, providing enterprise-grade security with support for multiple authentication providers. Upon successful authentication, a unique User ID (UID) is generated and securely stored in Firebase Firestore, serving as the primary anchor for all user-specific data, conversation histories, and personalization settings.

❖ Step 2: Session Initialization and Service Provisioning

The session establishment process involves multiple coordinated steps:

- **Unique Session Identification:** A `client_id` (UUID format) is generated to uniquely identify each browser session instance
- **WebRTC Token Acquisition:** ICE (Interactive Connectivity Establishment) tokens are requested from Azure services to enable peer-to-peer media streaming
- **Service Authentication:** Short-lived OAuth2 tokens are obtained for secure access to Azure Speech services, AI Agent, and other cloud resources
- **Real-Time Communication Setup:** Persistent WebSocket connections are established using Socket.IO for bidirectional, low-latency message exchange

❖ Step 3: Backend Validation and Security Controls

The backend implements comprehensive security measures including:

- Session integrity verification and client authentication
- Token lifecycle management (validation, scope verification, expiration handling)
- Secure header implementation and encrypted payload transmission
- Trust boundary enforcement between services without exposing sensitive credentials to the frontend.

2.6.3 Real-Time Voice Interaction Pipeline

❖ Step 1: Advanced Audio Capture and Streaming

The frontend captures microphone input through a sophisticated audio processing system:

- Continuous recording via MediaRecorder API with optimized codec selection
- Audio segmentation into precise 100-millisecond chunks for minimal latency

- Base64 encoding and compression for efficient transmission
- WebSocket streaming through persistent channels with automatic reconnection handling

❖ **Step 2: Intelligent Voice Activity Detection (VAD)**

Each audio chunk undergoes advanced processing through the Silero VAD neural network:

- PyTorch-based model computing speech probability scores with high accuracy
- Configurable threshold system (default: 0.5) adaptable to different acoustic environments
- Automatic noise filtering and silence detection to optimize processing resources
- Turn-taking logic that temporarily halts avatar speech to prevent conversational overlaps .

The complete technical implementation of the VAD system, including the VADIterator class and audio conversion utilities, is detailed in **Appendix A**.

❖ **Step 3: Real-Time Speech-to-Text Processing**

The STT pipeline implements cutting-edge speech recognition capabilities:

- Azure Speech-to-Text integration using PushAudioInputStream for continuous transcription
- Multi-language automatic detection with dynamic acoustic model selection
- Real-time streaming transcription with incremental result updates
- Context-aware processing that maintains conversational coherence across turns

❖ **Step 4: Advanced AI Agent Processing and Response Generation**

The transcribed text undergoes sophisticated natural language processing:

- **Language Detection:** Automated identification of user language with localization support
- **Context Management:** Conversation thread preservation with memory injection for coherent responses
- **AI Integration:** Azure AI Studio Agent leveraging OpenAI's large language models for intelligent response generation
- **Prompt Engineering:** Advanced prompting strategies ensuring consistent, relevant, and contextually appropriate outputs
- **Quality Assurance:** Content validation and appropriateness filtering before response delivery

❖ Step 5: SSML-Enhanced Speech Synthesis and Avatar Rendering

The response generation process includes multiple sophisticated stages:

- **SSML Formatting:** Advanced Speech Synthesis Markup Language formatting controlling pronunciation, pitch, tempo, prosodic emphasis, and emotional expression
- **High-Quality TTS:** Azure Text-to-Speech synthesis with neural voice models for natural-sounding speech
- **Avatar Integration:** Synchronized visual rendering with facial expressions, lip synchronization, and customizable character settings
- **WebRTC Delivery:** Real-time audio-visual streaming ensuring perfect synchronization between speech and visual elements

❖ Step 6: Comprehensive Logging and Analytics

Every interaction is meticulously tracked through a robust logging system:

- **Conversation Data:** Complete capture of voice input (original audio and transcription), AI responses, and interaction metadata
- **Performance Metrics:** Turn-level timing analysis, latency measurements, and system performance indicators
- **User Analytics:** Engagement patterns, session duration, interaction frequency, and user behavior analysis
- **Quality Monitoring:** Response quality assessment, error tracking, and system health monitoring

2.6.4 Flexible Text-Based Interaction Alternative

The system provides seamless multimodal interaction capabilities through a parallel text processing pipeline:

➤ Text Interaction Workflow

- **Input Processing:** Direct text input capture through the web interface with immediate WebSocket transmission
- **Pipeline Optimization:** Bypasses audio processing stages (VAD, STT) for reduced latency while maintaining conversation context
- **Unified AI Processing:** Identical AI Agent processing pipeline ensuring consistent response quality across modalities
- **Standard Output Pipeline:** Full SSML formatting, TTS synthesis, and avatar rendering maintaining visual consistency
- **Integrated Logging:** Comprehensive interaction logging with modality identification for analytics and optimization

2.6.5 Advanced Technical Optimizations and Performance Engineering

The system implements multiple optimization strategies for ultra-low latency:

- **Chunked Processing:** Real-time audio chunking and streaming for immediate response initiation
- **Asynchronous Architecture:** Non-blocking message processing with concurrent pipeline stages
- **Adaptive Thresholds:** Tunable VAD sensitivity for different acoustic environments and use cases
- **Intelligent Caching:** Prompt caching and memory optimization techniques for enhanced LLM performance
- **Progressive Rendering:** Response preview capabilities enabling user feedback before complete synthesis

➤ **Sophisticated Interruption and Concurrency Management**

- **Natural Turn-Taking:** VAD-driven interruption handling mimicking human conversation patterns
- **Thread Safety:** Mutex locks and atomic operations ensuring consistent state management across concurrent sessions
- **Graceful Degradation:** Comprehensive error handling for API timeouts, network failures, and service interruptions
- **Failover Mechanisms:** Automated fallback responses and service recovery procedures

➤ **Quality Assurance and Content Filtering**

- **Text Sanitization:** Advanced regex filtering and text cleaning for optimal speech synthesis
- **Content Moderation:** Real-time output validation ensuring appropriate and compliant responses
- **Recovery Systems:** Automatic re-synthesis capabilities for failed utterances and error correction
- **Monitoring Infrastructure:** Comprehensive anomaly detection for system latency and output quality assessment

2.6.6 Production Deployment

The deployment strategy follows industry best practices with complete containerization:

➤ **Multi-Stage Docker Architecture:**

- Optimized build processes with minimal image sizes and reduced attack surfaces

- Service decoupling through independent container deployment
- Security hardening following Azure security baseline recommendations

➤ **Automated CI/CD Pipeline:**

- **Quality Assurance:** Automated static analysis, security scanning, and comprehensive unit testing
- **Build Optimization:** Intelligent caching, layer optimization, and vulnerability assessment
- **Registry Management:** Secure image versioning and distribution through Azure Container Registry (ACR)

➤ **Azure Container Apps Production Hosting**

The production environment leverages Azure Container Apps for enterprise-grade hosting:

- **Auto-Scaling:** Dynamic scaling based on CPU utilization, memory consumption, and concurrent user metrics
- **Zero-Downtime Deployments:** Blue-green deployment strategy ensuring continuous service availability
- **Security Integration:** Azure Key Vault integration for secure secret management and environment-specific configuration
- **Network Security:** Ingress configuration with custom domain support, TLS encryption, and traffic management

2.7 Azure AI Foundry: Multi-Agent System Implementation

2.7.1 Platform Overview

Azure AI Foundry Agent Service is a platform that combines models, tools, frameworks, and governance into a unified system for building intelligent agents . It connects the core pieces of Azure AI Foundry such as models, tools, and frameworks into a single runtime, managing threads, orchestrating tool calls, enforcing content safety, and integrating with identity, networking, and observability systems to ensure agents are secure, scalable, and production-ready .

The service distinguishes itself by abstracting away infrastructure complexity and enforcing trust and safety by design, making it easy to move from prototype to production with confidence , enabling businesses to deploy intelligent automation that goes beyond simple chatbots to complete actual business goals.

➤ **Key Features:**

Production-Ready Architecture: Offers a fully managed cloud service with SDKs for Python, C#, and TypeScript, simplifying AI agent development and reducing complex tasks like function calling to just a few lines of code.

Enterprise Integration: Applies enterprise-grade trust features including identity via Microsoft Entra, RBAC, content filters, encryption, and network isolation .

Comprehensive Tooling: Agents can access enterprise knowledge (such as Bing, SharePoint, Azure AI Search) and take real-world actions (via Logic Apps, Azure Functions, OpenAPI, and more)

Full Observability: Captures logs, traces, and evaluations at every step with full thread-level visibility and Application Insights integration, allowing teams to inspect every decision and continuously improve agents over time

Model Flexibility: Works with a growing catalog of large language models including GPT-4o, GPT-4, GPT-3.5 (Azure OpenAI), and others like Llama

2.7.2 Multi-Agent Architecture Design

Our implementation employs a dual-agent architecture within Azure AI Foundry Agent Service to optimize both response speed and quality. The Question-Answer Agent handles initial query processing and knowledge retrieval, while the Reviewer Agent provides quality assurance and response enhancement. This collaborative approach ensures that users receive accurate, contextual, and well-refined responses through the AI avatar interface.

The architecture follows a sequential processing model where each agent contributes specialized capabilities. The Question-Answer Agent focuses on rapid information retrieval through vector search, while the Reviewer Agent applies advanced reasoning through GPT-4o integration to validate and enhance responses before delivery.

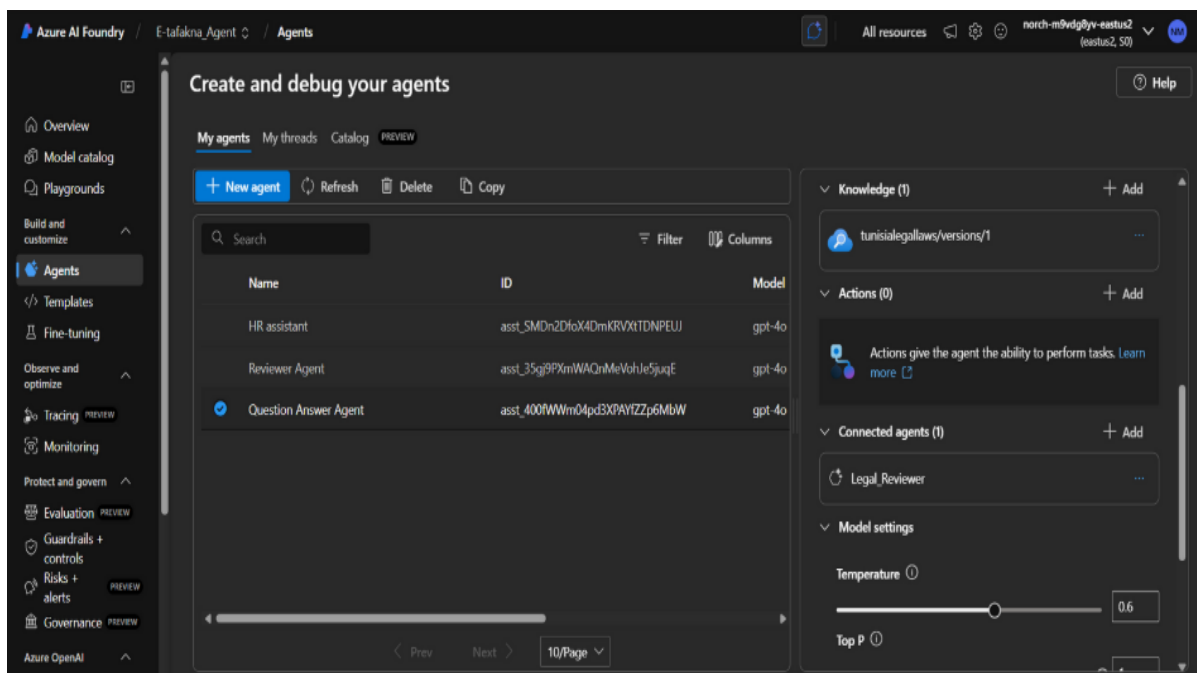


Figure 4. 8 : Azure AI Foundry Agent Creation

2.7.3. Question-Answer Agent Implementation

The Question-Answer Agent serves as the primary response generator in our system. Upon receiving user queries, this agent performs semantic searches across configured knowledge bases using vector search capabilities. The agent processes retrieved information within conversation context, leveraging Azure AI Foundry's thread management to maintain continuity across interactions.

The agent incorporates automatic language detection to support multilingual interactions, particularly French, English, and Arabic. This ensures that responses are culturally appropriate and linguistically accurate. The agent maintains conversation history through thread-based memory, enabling contextual understanding and follow-up question handling.

2.7.4 Reviewer Agent Implementation

The Reviewer Agent operates as a quality assurance layer that validates and enhances responses from the Question-Answer Agent. It employs a dual-processing approach, combining vector search for context verification with GPT-4o integration for advanced reasoning and language optimization.

This agent performs secondary validation searches to ensure response accuracy and completeness. The GPT-4o integration applies sophisticated prompt engineering to enhance response clarity, coherence, and user engagement. The Reviewer Agent ensures that final responses meet enterprise standards for accuracy while maintaining natural conversational flow.

➤ Integrated Tool Ecosystem

AI agents are powered by several key capabilities that make them much more dynamic and versatile than traditional systems. For example, agents can use tools such as web search, code execution, and calendars, while retaining memory from past interactions to inform their decisions.

The service provides a comprehensive range of built-in tools including:

- **Grounding with Bing Search** for real-time information access
- **Azure AI Search** for vector and semantic search within knowledge bases
- **Azure Logic Apps** for business workflow integration
- **Third-party partner tools** like TripAdvisor for specialized functionalities
- **Custom integrations** via Function Calling to extend agent capabilities, as well as built-in tools including Fabric, SharePoint, Azure AI Search, and Azure Storage for rapid development

2.8 Azure Speech Services Implementation

➤ Overview

The Real-Time AI Voice Agent System leverages three core Azure Cognitive Services to create an immersive conversational experience. These services work together seamlessly to provide natural voice-to-voice interaction with an AI assistant through an animated avatar interface, forming a complete end-to-end voice communication pipeline.

2.8.1 *Speech-to-Text (STT) Service*

➤ Service Description

The Speech-to-Text service serves as the primary input mechanism for the voice agent system, converting spoken audio into written text in real-time. This service enables users to communicate naturally with the AI assistant using their voice, eliminating the need for traditional text-based input methods. The STT service acts as the first critical component in the voice interaction pipeline, capturing and processing human speech with high accuracy and minimal latency.

➤ Technical Implementation Features

The STT implementation supports multiple languages including French, English, and Arabic, with automatic language detection capabilities that ensure appropriate response formatting. The system utilizes continuous recognition mode, allowing for uninterrupted conversation flow without requiring users to manually start and stop recording sessions. Real-time audio streaming is implemented through WebSocket connections, providing low-latency communication between the client and Azure's speech recognition endpoints.

The service incorporates Voice Activity Detection (VAD) using the Silero VAD model, which intelligently identifies when users are speaking and automatically interrupts the avatar's speech output when voice activity is detected. This creates a natural conversation flow similar to human-to-human interactions. The STT service also measures and reports speech recognition latency, providing performance metrics that ensure optimal user experience.

2.8.2 *Text-to-Speech (TTS) Service*

➤ Service Description

The Text-to-Speech service transforms the AI assistant's text responses into natural-sounding speech output, completing the voice interaction loop. This service uses advanced neural voice synthesis technology to generate human-like speech that maintains consistent tone, pronunciation, and emotional context throughout the conversation. The TTS service supports multiple voice profiles and can be customized to match different personality characteristics or regional accents.

➤ **Advanced Voice Synthesis Features**

The TTS implementation utilizes Azure's neural voice technology, specifically the "en-US-AvaMultilingualNeural" voice model, which provides high-quality, natural-sounding speech output across multiple languages. The service supports Speech Synthesis Markup Language (SSML) for fine-grained control over speech characteristics including pronunciation, speaking rate, pitch, and emphasis. Custom voice profiles can be integrated through speaker profile IDs, allowing for personalized voice experiences.

The system implements intelligent text preprocessing to clean and optimize content for speech synthesis. This includes removing special formatting characters, emojis, and markdown elements that are not suitable for verbal communication. The service also handles sentence-level punctuation detection to create natural speaking patterns with appropriate pauses and intonation.

➤ **Real-time Speech Queue Management**

A sophisticated queue management system ensures smooth speech delivery by processing text chunks as they are generated by the AI assistant. This streaming approach minimizes perceived latency by beginning speech synthesis before the complete response is generated. The system handles interruption scenarios gracefully, allowing users to interrupt the avatar's speech naturally without audio artifacts or delays.

2.8.3 Animated Avatar Service

➤ **Service Description**

The Animated Avatar service provides a visual representation of the AI assistant through a realistic, animated character that synchronizes lip movements, facial expressions, and gestures with the generated speech. This service creates an engaging, human-like interaction experience that enhances user engagement and makes the conversation feel more natural and intuitive. The avatar serves as the visual embodiment of the AI assistant, providing non-verbal communication cues that complement the voice interaction. As illustrated in Figure 2.9, the animated avatar interface demonstrates the real-time visual feedback system integrated with the speech services.



Figure 4. 9: Animated Avatar Visual Representation

➤ **Visual and Animation Capabilities**

The avatar system supports multiple character models and styles, with the current implementation featuring a professional business-style character named "Meg." The service provides customizable visual settings including background colors, transparency options, and video cropping capabilities to fit different interface requirements. The avatar's facial animations are automatically synchronized with the TTS output, creating realistic lip-sync and facial expressions that match the spoken content.

The system utilizes WebRTC technology for real-time video streaming, ensuring low-latency visual feedback that maintains synchronization with the audio output. Video quality and bitrate can be adjusted based on network conditions and user preferences, with support for different resolution outputs ranging from cropped views focusing on the face to full-body presentations.

➤ **Integration with Communication Pipeline**

The Animated Avatar service is seamlessly integrated with both the TTS service and the overall communication architecture through WebSocket connections and real-time signaling protocols. The avatar begins animating as soon as speech synthesis starts, creating natural timing that mirrors human conversation patterns. The service handles connection management, including automatic reconnection capabilities and graceful degradation when network conditions are suboptimal.

The avatar system supports real-time control messages for starting, stopping, and modifying animations, allowing for responsive interaction management. This includes the ability to interrupt avatar speech and animations when users begin speaking, maintaining the natural flow of conversation and preventing overlapping communication that could confuse or frustrate users.

2.9 Evaluation

➤ **Previous Issues and Current Resolution**

During the initial testing phase, the system encountered certain challenges with complex conditional legal reasoning, particularly in inheritance law scenarios. However, these issues have been systematically identified and corrected through iterative system improvements and enhanced legal knowledge base refinement.

➤ **Case Study: Inheritance Law Correction Example**

Previous System Limitation (Now Resolved): In earlier versions, the system occasionally struggled with applying conditional legal logic based on specific family circumstances in inheritance cases.

Test Query: *"My father passed away and he has a wife who has no children from him. Do I have to give her part of my dad's land?"*

Previous Incorrect Response (Historical): *"According to Article 101, the wife inherits $\frac{1}{4}$ or $\frac{1}{2}$ of the estate."*

Current Corrected Response: *"Based on your father's family situation, according to **Article 102 of the Tunisian Personal Status Code**, since your father had no children from his wife, she is entitled to inherit $\frac{1}{4}$ (**one-quarter**) of **his estate**, including any land property. This inheritance right is mandatory under Tunisian law and cannot be denied. You and other legal heirs will inherit the remaining $\frac{3}{4}$ of the estate according to the prescribed inheritance shares. It's important to note that if your father had children from his wife, her share would have been reduced to $\frac{1}{8}$ (**one-eighth**) of the estate."*

➤ System Enhancement Details

Improved Legal Logic Processing: The system now correctly identifies and applies conditional inheritance rules based on family composition. The enhanced algorithm properly distinguishes between different inheritance scenarios and references the appropriate legal articles with precise conditions.

Context-Aware Response Generation: The current system demonstrates improved contextual understanding by:

- Correctly identifying the absence of children as a determining factor
- Applying the appropriate legal article (Article 102 instead of Article 101)
- Providing the exact inheritance fraction ($\frac{1}{4}$) based on the specific circumstances
- Explaining the conditional nature of inheritance shares
- Offering additional clarification about alternative scenarios

Enhanced Legal Knowledge Integration: The system now maintains a more sophisticated understanding of:

- Conditional legal provisions and their specific applications
- The relationship between different legal articles and their appropriate contexts
- The importance of family structure in determining inheritance rights
- The mandatory nature of inheritance laws in the Tunisian legal system

3. Conclusion

The modeling efforts resulted in a high-performing Vector RAG system capable of processing complex Tunisian legal documents while maintaining context, hierarchy, and semantic relationships. Conditional legal scenarios, such as inheritance rules, are now accurately handled, demonstrating the system's improved reasoning and response quality. This robust modeling foundation naturally leads to the deployment phase, detailed in the following chapter, where the system will be transitioned into a production-ready environment for real-time legal assistance.

Chapter V: DEPLOYMENT

1. Introduction

This chapter presents the deployment phase of our AI legal assistant system, focusing on transforming the developed models and interfaces into a fully operational, production-ready environment. It begins with an overview of the user-facing components, including the registration, login, and AI consultation interfaces designed to ensure accessibility, security, and user trust. The chapter also details the technical implementation of the frontend, the integration of multimodal communication, and the role of data visualization in monitoring platform usage. Finally, it explains the deployment process using modern DevOps practices, including containerization, continuous integration, and cloud-based hosting on Microsoft Azure, ensuring scalability, reliability, and secure access for real-time legal assistance.

2. Deployment

2.1 Deployment Phase Overview

We developed a comprehensive React-based frontend application for our AI legal assistant. This phase focuses on creating user-friendly interfaces that enable seamless interaction with our intelligent legal consultation system. The application consists of three main interfaces designed to provide a complete user experience from authentication to active legal consultation.

2.2 User Registration Interface

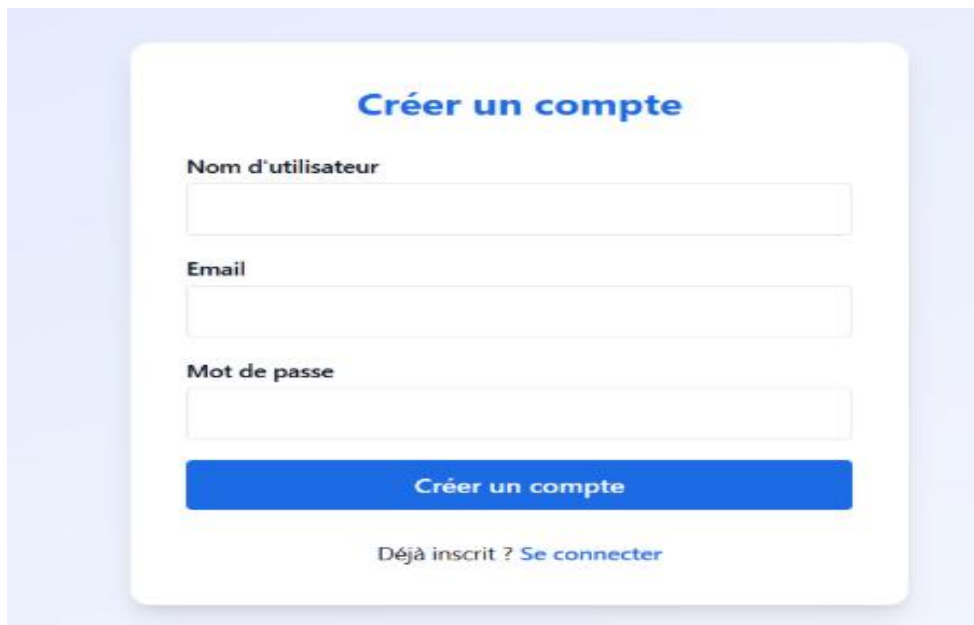
The image shows a user registration form titled "Créer un compte" in blue. It features three input fields: "Nom d'utilisateur", "Email", and "Mot de passe". Below these fields is a blue button labeled "Créer un compte". At the bottom, there is a link that says "Déjà inscrit ? Se connecter". The form is centered on a light blue background.

Figure 5. 1: User Account Creation Interface

The registration interface illustrated in Figure 5.1 presents a clean, professional onboarding experience for new users. The form features three essential fields: username

("Username"), email, and password ("Password"), implemented with a minimalist design approach that builds trust and reduces user anxiety. The prominent blue "Create Account" button follows modern UI/UX principles, while the "Already registered? Sign in" link ensures smooth navigation between authentication states. This interface establishes the foundation for secure user access to our AI legal consultation platform

2.3 User Login Interface

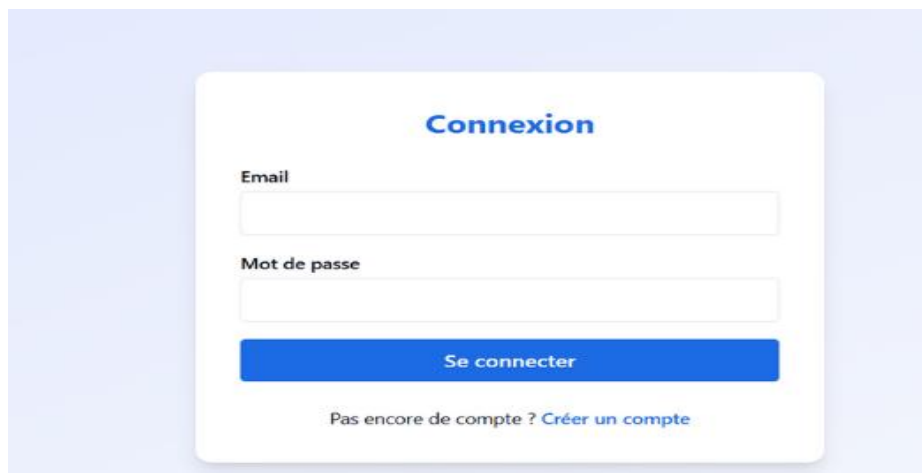


Figure 5. 2: User Authentication Interface

The login interface illustrated in Figure 5.2 provides returning users with streamlined access through a simplified two-field design. Users enter their email and password ("Password") credentials through an intuitive form that maintains visual consistency with the registration interface. The "Sign In" button enables quick authentication, while the "No account yet? Create an account" link facilitates new user acquisition. This design prioritizes user convenience while maintaining security standards essential for legal applications.

2.4 AI Legal Consultation Interface

The main consultation interface illustrated in Figure 5.3 represents the core innovation of our platform, featuring the 'Meg' avatar as an AI legal consultant. This sophisticated interface integrates multiple interactive components: a 10-minute session management system with Start/Pause/Terminate controls and timer display, a left panel offering language selection ("French") and real-time transcription capabilities ("Real-time transcript"), and a comprehensive chat interface supporting both typed and spoken legal questions ("Type your message or speak to the avatar..."). The right panel provides step-by-step user guidance ("How to use"), while the professional navigation header reinforces the platform's 24/7 availability ("Your virtual legal advisor available 24/7"). This interface successfully combines advanced AI technology with intuitive user interaction design, creating an accessible yet professional legal consultation environment.

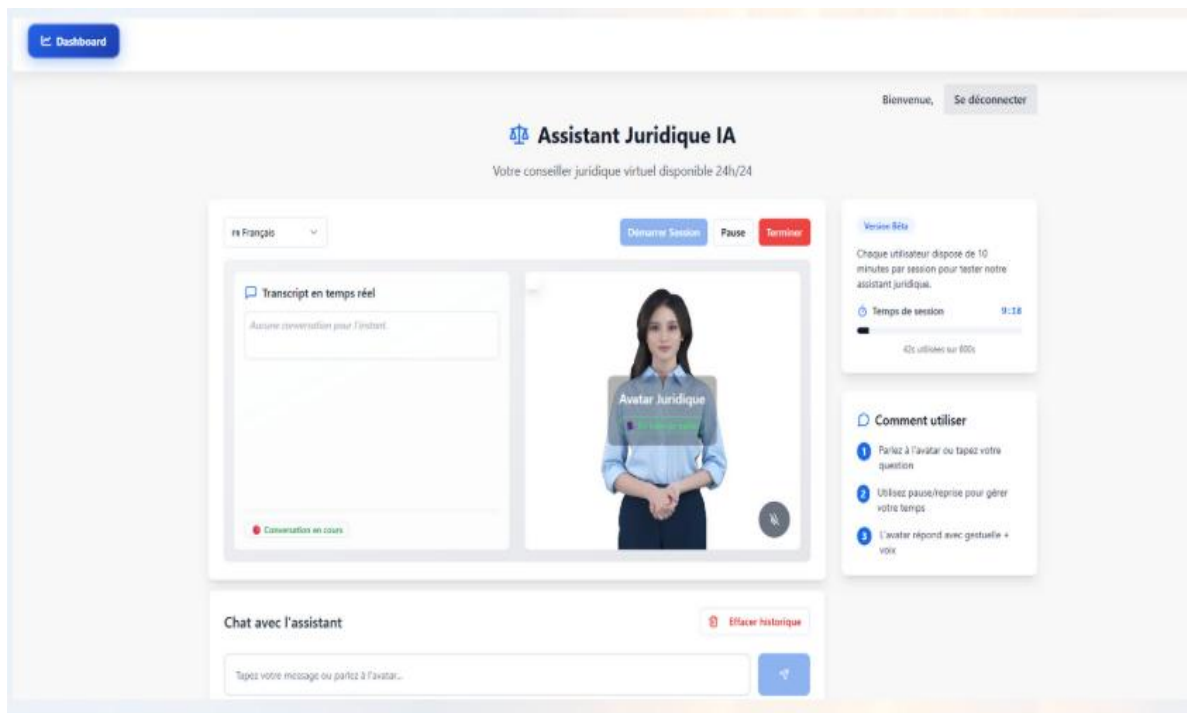


Figure 5. 3 AI Legal Assistant Main Interface with Avatar Technology

2.5 Technical Implementation Impact

Our React-based frontend implementation demonstrates successful integration of modern web technologies with AI-powered legal services. The responsive design ensures cross-device accessibility, while the avatar-mediated interface enhances user trust and engagement. The multi-modal communication system (text and voice) accommodates diverse user preferences, and the real-time transcription feature provides valuable consultation documentation. This comprehensive interface design positions our AI legal assistant as an innovative solution for accessible legal consultation services.

2.6. Data Visualization Overview

The transition from raw data collection to meaningful business insights occurs through a comprehensive visualization layer that transforms Firebase data streams into actionable analytics. The dashboard serves as the central hub where real-time user interactions, system performance metrics, and engagement patterns converge into visual representations that enable data-driven decision making. This visualization approach ensures that stakeholders can quickly identify trends, monitor system health, and understand user behavior patterns without diving into complex database queries or technical logs.

➤ Complete KPI Inventory

The AI Avatar Dashboard tracks **13 distinct Key Performance Indicators** across four main categories:

- **User Engagement KPIs (5)**
 - **Total Users** - Overall user acquisition and platform growth
 - **Total Messages** - Cumulative interaction volume measurement
 - **Total Conversations** - Conversation thread creation and depth tracking
 - **Active Now** - Real-time user presence and concurrent usage
 - **Engagement Rate** - User-to-conversation ratio indicating platform stickiness
- **Real-time System KPIs (4)**
 - **Messages/Min** - Live activity pulse and usage intensity
 - **Error Rate** - System reliability and user experience quality
 - **Voice Interactions** - Current voice feature adoption
 - **Text Interactions** - Current text interface usage
- **Trend Analysis KPIs (3)**
 - **Daily Messages Trend** - Historical usage patterns and growth trajectory
 - **Hourly Usage Pattern** - Peak time identification and resource optimization
 - **Voice vs Text Usage Evolution** - Feature adoption trends over time
- **Activity Monitoring KPI (1)**
 - **Recent Activities Feed** - Live user behavior and system event stream

Table 5. 1: Core Business Metrics and User Engagement Indicators

Metric	Formula/Source	Description	Business Utility
Total Users	len(users) from Firebase collection	Total number of registered users in the system	User Growth Tracking: Monitor user acquisition and overall platform adoption
Total Messages	sum(messages) across all Firebase threads	Cumulative count of all messages sent in the platform	Engagement Volume: Measure overall platform activity and user interaction levels
Total Conversations	count(threads) from Firebase user collections	Total number of conversation threads created	Interaction Depth: Track conversation initiation and user engagement patterns
Active Now	len([ctx for ctx in client_contexts.values() if ctx.get('user')])	Number of users currently connected/online	Real-time Activity: Monitor current platform usage and peak activity times
Engagement Rate	(total_conversations / total_users) * 100	Percentage showing conversation-to- user ratio	User Quality: Measure how actively users engage with the AI avatar

Table 5. 2 : Real-time System Performance and Activity Indicators

Metric	Formula/Source	Description	Business Utility
Messages/Min	len(realtime_metrics['recent_activities'][-5:])	Rate of messages sent in the last minute	Activity Pulse: Monitor real-time platform activity and usage spikes
Error Rate	(error_count / max(api_calls_count, 1)) * 100	Percentage of failed operations	System Health: Track system reliability and user experience quality
Voice Interactions	sum(1 for ctx in client_contexts.values() if ctx.get('speech_recognizer'))	Current users using voice features	Feature Adoption: Monitor voice feature usage and multimodal engagement
Text Interactions	current_active - voice_interactions	Current users using text-only interface	Interface Preference: Track user preference between voice and text modes

Table 5. 3: Daily Message Volume Trend Analysis

Component	Formula/Source	Description	Business Utility
Chart Data	firebase_stats['daily_messages'] - 14-day rolling window	Line chart showing message volume over time	Trend Analysis: Identify usage patterns, growth trends, and seasonal variations

Table 5. 4: User Activity Distribution and Interaction Patterns

Component	Formula/Source	Description	Business Utility
Chart Data	Voice/Text/Conversations/Active Users distribution	Pie chart showing breakdown of different activity types	Usage Pattern Analysis: Understand how users interact with different platform features

Table 5. 5: Hourly Platform Usage and Peak Time Analysis

Component	Formula/Source	Description	Business Utility
Chart Data	firebase_stats['hourly_usage'] - 24-hour activity pattern	Bar chart showing usage by hour of day	Peak Time Identification: Optimize resource allocation and maintenance

Table 5. 6: Multimodal Interaction Feature Adoption Tracking

Component	Formula/Source	Description	Business Utility
Voice Data	message.get('is_voice', False) == True count over time	Line showing voice interaction trends	Feature Evolution: Track adoption of voice features over time
Text Data	message.get('is_voice', False) == False count over time	Line showing text interaction trends	Interface Preference: Monitor text vs voice usage balance

Table 5. 7: Real-time User Activity and System Event Monitoring

Component	Formula/Source	Description	Business Utility
Recent Activities	realtime_metrics['recent_activities'] - Last 15 activities	Live stream of recent user actions and system events	Real-time Monitoring: Track user behavior and system events as they happen

2.6.1 Dashboard Visualizations

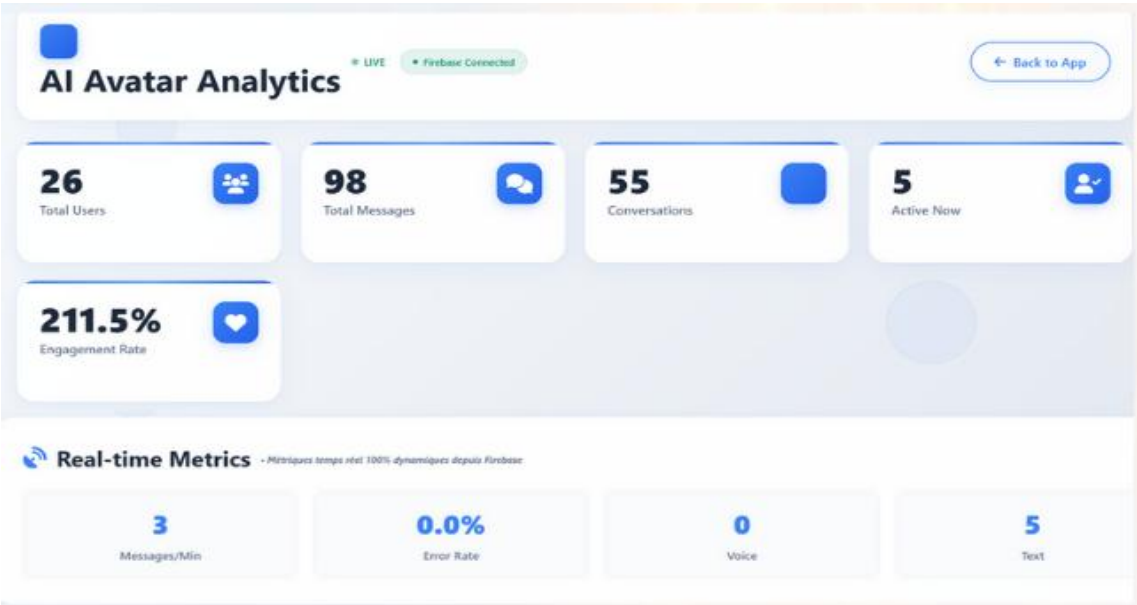


Figure 5. 4: Main Dashboard Overview with Core KPIs

Our analytics dashboard illustrated in Figure 5.4 displays real-time metrics from Firebase database: 26 total users, 98 messages, 55 conversations, with 5 active users. The 211.5% engagement rate (conversations divided by users) indicates multiple conversation sessions per user. Real-time metrics show messages per minute, error rates, and voice versus text interaction breakdown.

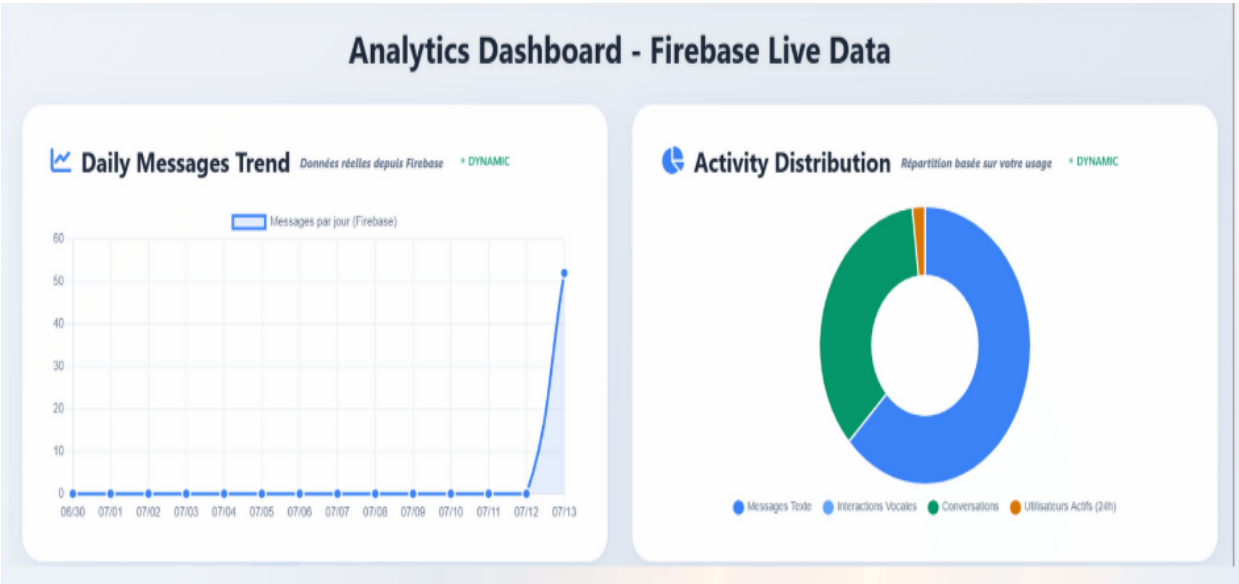


Figure 5. 5: Daily Trends and Activity Distribution Analytics

Our Firebase dashboard illustrated in Figure 5.5 features two visualizations: Daily Messages Trend showing growth from zero to 50+ messages indicating strong adoption, and

Activity Distribution with text messages dominating usage. Both provide live data updates for immediate platform performance insights.



Figure 5. 6: Usage Patterns and Communication Preferences Analysis

Our dashboard illustrated in Figure 5.6 includes Hourly Usage Pattern and Voice vs Text Usage Over Time charts. The hourly pattern shows peak activity at 9-10h with 23 interactions. The voice vs text chart demonstrates text messages surging to 55 interactions while voice interactions remain minimal

2.7. Complete Application Deployment

2.7.1 Tools Definition

 **GitHub Actions**



Figure 5. 7: Logo GitHub Actions

Definition: GitHub Actions is a CI/CD platform integrated with GitHub that automatically automates build, testing, and deployment processes as soon as code is pushed or a pull request is opened. This powerful automation tool eliminates manual intervention by creating workflows that respond to Git events and execute predefined sequences of tasks including code checkout, dependency installation, testing, building, and deployment.

Key Features: The platform uses YAML-based workflow definitions stored in the repository, providing version-controlled automation scripts. It offers seamless integration with GitHub repositories, extensive marketplace of pre-built actions, and supports multiple programming languages and deployment targets.

Docker (Dockerfile)



Figure 5. 8: Logo Docker Containerization

Definition: Docker is a containerization platform that allows applications to be packaged into lightweight, portable containers. A Dockerfile serves as a blueprint that describes how to bundle the application along with its dependencies, runtime environment, and configuration into a single executable image. This approach ensures consistent behavior across different environments from development to production.

Benefits: Docker containers provide isolation, portability, and reproducibility. They eliminate the "it works on my machine" problem by encapsulating the entire application stack, including the operating system libraries, runtime, and dependencies, into a single deployable unit.

Azure Container Registry (ACR)



Azure Container Registry

Figure 5. 9 Logo Azure Container Registry Service

Definition: Azure Container Registry is a private, managed Docker registry service hosted on Microsoft Azure. It provides secure storage, versioning, and distribution of container images while following the Open Container Initiative (OCI) specifications. ACR serves as a central repository for managing Docker images with enterprise-grade security and access controls.

Advanced Capabilities: The service includes automated build triggers through ACR Tasks, geographic replication for global distribution, integrated security scanning for vulnerability detection, and role-based access control (RBAC) for fine-grained permissions management.

Azure Container Apps

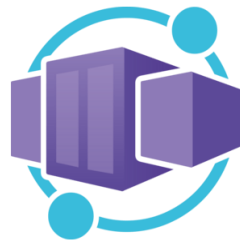


Figure 5. 10: Logo Azure Container Registry Service

Definition: Azure Container Apps is a fully managed serverless platform designed for running containerized applications without the complexity of managing underlying infrastructure. It provides automatic scaling, load balancing, and pay-per-use billing model, making it ideal for microservices and event-driven applications.

Scaling Intelligence: The platform leverages KEDA (Kubernetes Event-Driven Autoscaler) to implement intelligent auto-scaling based on various metrics including HTTP requests, CPU utilization, memory consumption, and custom events. The revolutionary scale-to-zero capability ensures cost optimization by completely shutting down unused applications.

2.7.2 Deployment Process

Phase 1: Project Foundation and Setup

The deployment journey begins with establishing a solid foundation for your Python application. This phase involves organizing your project structure with all necessary configuration files, dependencies, and documentation. The application code should be properly structured with clear separation of concerns, including main application files, requirements specifications, and containerization blueprints. During this stage, developers ensure that the application is production-ready with proper error handling, logging, and health check endpoints.

The repository structure becomes crucial as it defines how the automated systems will interact with your code. This includes creating appropriate directories for source code, configuration files, tests, and deployment scripts. The foundation phase also involves setting

up proper version control practices and branching strategies that will support the automated deployment pipeline.

Phase 2: Azure Cloud Infrastructure Preparation

The second phase focuses on establishing the cloud infrastructure components required for hosting the containerized application. This involves provisioning Azure resources including resource groups, container registries, and container app environments. The infrastructure setup follows Azure best practices for security, scalability, and cost optimization.

During this phase, administrators configure the Azure Container Registry with appropriate access policies, security scanning capabilities, and geographic replication if needed for global distribution. The container app environment is prepared with networking configurations, monitoring capabilities, and integration with other Azure services. This foundational infrastructure ensures that the application will have a robust, secure, and scalable hosting environment.

Phase 3: Continuous Integration and Deployment Pipeline

The automation pipeline represents the heart of the DevOps workflow, transforming code commits into live applications. This phase involves configuring GitHub Actions workflows that automatically trigger on code changes, executing a series of well-orchestrated steps including code validation, testing, building, and deployment. The pipeline ensures quality through automated testing while maintaining rapid deployment cycles.

The workflow orchestrates complex interactions between different tools and services, handling authentication, image building, registry operations, and application deployment. Error handling and rollback mechanisms are integrated to ensure reliability and minimize downtime. The pipeline also includes notification systems to keep development teams informed about deployment status and any issues that may arise.

Phase 4: Security and Access Management

Security configuration forms a critical phase where authentication credentials, service principals, and access policies are established. This involves creating secure connections between GitHub Actions and Azure services through service principals and managed identities. Secret management ensures that sensitive information like passwords, API keys, and connection strings are properly protected and accessed only by authorized systems.

Role-based access control (RBAC) is configured to provide principle of least privilege access across all components. Security scanning is enabled at multiple levels including container image vulnerability assessment and runtime security monitoring. This comprehensive security approach protects the application and infrastructure from potential threats while maintaining operational efficiency.

Phase 5: Application Deployment and Configuration

The deployment phase brings together all previous preparations to launch the application in the production environment. Container apps are configured with appropriate resource allocations, networking settings, and scaling parameters. The deployment process includes health checks, readiness probes, and gradual rollout strategies to ensure smooth transitions.

Configuration management ensures that environment-specific settings are properly applied without hardcoding sensitive values. The deployment system handles traffic routing, SSL certificate management, and domain configuration to make the application accessible to end users. Monitoring and logging are activated to provide visibility into application performance and behavior.

Phase 6: Monitoring, Scaling, and Optimization

The final phase establishes ongoing operational excellence through comprehensive monitoring, intelligent scaling, and continuous optimization. Auto-scaling rules are configured based on application-specific metrics and business requirements. The system monitors performance indicators, resource utilization, and user experience metrics to ensure optimal operation.

Cost optimization strategies are implemented including scale-to-zero configurations for development environments and intelligent resource allocation for production workloads. Performance monitoring provides insights for further optimization opportunities while alerting systems notify administrators of any issues requiring attention.

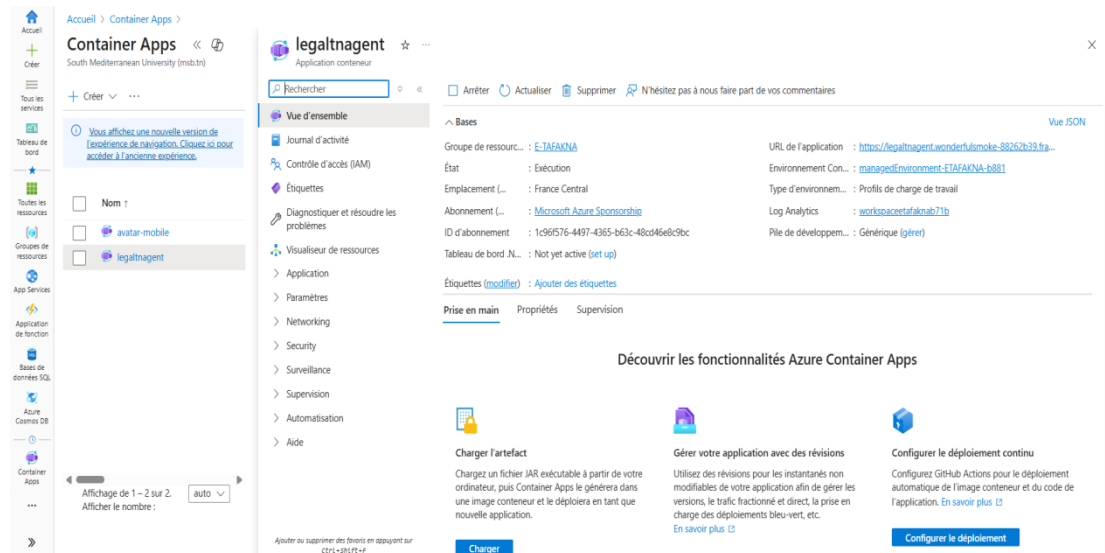


Figure 5. 11: Azure Container Apps Configuration Interface in Microsoft Azure Portal

3. Conclusion

The deployment phase successfully translated design and modeling efforts into a fully functional platform, combining a user-friendly React-based frontend with robust cloud infrastructure. Through secure authentication, multimodal consultation, and real-time analytics, the system demonstrates both accessibility and operational reliability. The integration of GitHub Actions, Docker, and Azure services established a scalable CI/CD pipeline, while monitoring and security measures ensure long-term stability. This comprehensive deployment provides the foundation for continuous improvement and sets the stage for the system's evaluation and performance analysis in the subsequent chapter.

GENERAL CONCLUSION

In a context marked by the growing necessity for accessible legal information and the modernization of legal services delivery, this project represents a comprehensive approach to transforming legal consultation through artificial intelligence innovation within E-TAFAKNA's ecosystem. The proposed solution aims to revolutionize both the accessibility of Tunisian legal knowledge and the consultation experience itself, through the development of an advanced AI legal assistant featuring multimodal interaction capabilities.

We have successfully developed an AI-powered legal avatar equipped with four key technological innovations: a professional 3D avatar with synchronized speech and lip movements, trilingual voice interaction capabilities supporting Arabic, French, and English, specialized Tunisian legal intelligence trained on local legal frameworks, and a comprehensive real-time analytics dashboard. Additionally, we designed a complete React-based frontend application integrated with Flask backend services, incorporating advanced Azure cloud services including AI Search, Speech Services, OpenAI integration, and AI Foundry multi-agent architecture. Furthermore, an interactive analytics dashboard was implemented for monitoring user engagement, system performance, and consultation patterns, alongside a comprehensive authentication and chat history management system powered by Firebase.

The project implementation was structured around five main phases following the TDSP methodology: **Project General Framework, Business Understanding, Data Acquisition & Understanding, Modeling, and Deployment**. This lifecycle ensured a systematic progression from identifying business objectives and collecting Tunisian legal data to designing intelligent models and deploying them within a secure, scalable infrastructure.

This project distinguishes itself through both technical and functional advantages. From a functional perspective, it addresses concrete needs in legal information accessibility, administrative guidance, and user experience improvement, serving both legal professionals and general citizens seeking legal consultation. From a technical standpoint, it combines custom AI agent development, intelligent tool integration, and the adoption of best practices in cloud deployment and testing, ensuring a scalable, coherent solution aligned with Tunisia's digital transformation objectives in the legal sector.

On a professional level, this project allowed us to strengthen our competencies in AI agent development and customization within Azure AI Foundry, artificial intelligence integration through advanced conversational systems, as well as data visualization through comprehensive analytics dashboards for real-time monitoring. From a personal perspective, this experience enabled us to develop our organizational skills, our ability to work in interdisciplinary teams, and to collaborate with diverse professional profiles including legal experts, AI engineers, and frontend developers.

Finally, several improvement perspectives can be envisioned, particularly the integration of advanced legal document processing capabilities to centralize and automate legal research workflows. We also plan the future integration of additional modules such as contract analysis

for more efficient client document review, and document generation automation, with the goal of optimizing legal service delivery and strengthening client relationships.

Our future vision includes global expansion to Saudi Arabia, France, and Morocco, with upcoming features for contract analysis, document generation, and compliance support, transforming E-TAFAKNA into a comprehensive global legal AI platform. This strategic expansion will leverage the modular technical architecture developed, enabling adaptation to local legal specificities while maintaining core technological innovations, positioning Tunisia as an innovative leader in international LegalTech and establishing the foundation for a profound transformation of legal services at regional and international scales.

WEBOGRAPHY

- [1] E-TAFAKNA. Official Company Website. Available at: <https://www.e-tafakna.com/> (Accessed: 20 February 2025).
- [2] Microsoft Learn. Team Data Science Process Lifecycle. Available at: <https://learn.microsoft.com/th-th/azure/architecture/data-science-process/lifecycle> (Accessed: 25 February 2025).
- [3] Data Science PM. TDSP Framework Guide. Available at: <https://www.datascience-pm.com/tdsp/> (Accessed: 25 February 2025).
- [4] Microsoft Azure. Microsoft-TDSP Documentation. Available at: <https://github.com/Azure/Microsoft-TDSP/blob/master/Docs/lifecycle-detail.md> (Accessed: 26 February 2025).
- [5] Microsoft Learn. What is Azure AI Foundry Agent Service? Available at: <https://learn.microsoft.com/en-us/azure/ai-foundry/agents/overview> (Accessed: 15 March 2025).
- [6] Microsoft Azure. Azure AI Foundry Agent Service Product Page. Available at: <https://azure.microsoft.com/en-us/products/ai-agent-service> (Accessed: 15 March 2025).
- [7] Microsoft Learn. Introduction to Azure AI Search. Available at: <https://learn.microsoft.com/en-us/azure/search/search-what-is-azure-search> (Accessed: 18 April 2025).
- [8] Microsoft Learn. Azure Cognitive Services Speech Documentation. Available at: <https://docs.microsoft.com/en-us/azure/cognitive-services/speech-service/> (Accessed: 25 April 2025).
- [9] Microsoft Learn. Azure OpenAI Service Documentation. Available at: <https://docs.microsoft.com/en-us/azure/cognitive-services/openai/> (Accessed: 25 April 2025).
- [10] GitHub Docs. GitHub Actions Documentation. Available at: <https://docs.github.com/en/actions> (Accessed: 25 March 2025).
- [11] Microsoft Learn. Azure Container Registry Documentation. Available at: <https://learn.microsoft.com/en-us/azure/container-registry/> (Accessed: 2 April 2025).
- [12] Microsoft Learn. Azure Container Apps Documentation. Available at: <https://learn.microsoft.com/en-us/azure/container-apps/> (Accessed: 8 April 2025).
- [13] Docker Inc. Docker Official Documentation. Available at: <https://docs.docker.com/> (Accessed: 12 April 2025).
- [14] Microsoft Corporation. Visual Studio Code. Available at: <https://code.visualstudio.com/> (Accessed: 20 April 2025).

- [15] Python Software Foundation. Python Programming Language. Available at: <https://www.python.org/> (Accessed: 20 April 2025).
- [16] Meta Platforms Inc. React - The Library for Web and Native User Interfaces. Available at: <https://react.dev/> (Accessed: 22 April 2025).
- [17] Pallets Project. Flask - Web Development, One Drop at a Time. Available at: <https://flask.palletsprojects.com/> (Accessed: 22 April 2025).
- [18] Google LLC. Firebase Documentation. Available at: <https://firebase.google.com/docs> (Accessed: 30 April 2025).
- [19] Economic Times Government. AI-backed SUVAS translation tool intended to make legalese simpler, court proceedings faster: Law Minister. Available at: <https://government.economictimes.indiatimes.com/news/technology/ai-backed-suvas-translation-tool-intended-to-make-legalese-simpler-court-proceedings-faster-law-minister/102648151> (Accessed: 15 February 2025).
- [20] LawTech Asia. Regulation of Legal Technology in Singapore: Issues, International Scans and Ideas. Available at: <https://lawtech.asia/regulation-of-legal-technology-in-singapore-issues-international-scans-and-ideas/> (Accessed: 15 February 2025).
- [21] Microsoft Learn. Azure AI Services Documentation. Available at: <https://docs.microsoft.com/en-us/azure/ai-services/> (Accessed: 28 April 2025).
- [22] KEDA. Kubernetes Event Driven Autoscaler Documentation. Available at: <https://keda.sh/docs/> (Accessed: 15 April 2025).

Appendix A: VAD (Voice Activity Detection) Implementation Code

This appendix contains the complete implementation of the VAD (Voice Activity Detection) iterator class used in this study. The code is primarily based on the Silero VAD model implementation (<https://github.com/snakers4/silero-vad>) and includes utility functions for audio format conversion.

A.1 VAD Iterator Class and Utility Functions

```
import copy
import torch
import numpy as np
class VADIterator:
    def __init__(
        self,
        model,
        threshold: float = 0.5,
        sampling_rate: int = 16000,
        min_silence_duration_ms: int = 100,
        speech_pad_ms: int = 30,
    ):
        """
        Mainly taken from https://github.com/snakers4/silero-vad
        Class for stream imitation
        Parameters
        -----
        model: preloaded .jit/.onnx silero VAD model
        threshold: float (default - 0.5)
```

- **Padding:** Adds padding to speech chunks for better context
- **State management:** Maintains internal states for continuous processing

The utility functions `int2float()` and `float2int()` provide audio format conversion between integer and floating-point representations commonly used in audio processing pipelines.

Abstract

This report is part of the graduation project at the host organization E-TAFAKNA to obtain the national diploma in computer engineering. In the digital era, Tunisia's legal landscape faces increasing complexity due to significant legislative activity and structural reforms, creating barriers for citizens, entrepreneurs, and legal professionals seeking accessible legal information. Traditional legal consultation methods involve scattered regulations, overlapping procedures, and inconsistent structures across legal texts, translating into tangible costs in terms of time and reliance on intermediaries.

Our project consists of developing a revolutionary Legal AI Avatar that integrates advanced artificial intelligence capabilities, including 3D avatar visualization, multilingual voice interaction, and specialized Tunisian legal intelligence, to transform traditional legal consultation into an accessible, engaging, and highly accurate service. The system addresses three critical limitations: limited legal accuracy for the Tunisian context, text-only communication barriers, and missing voice accessibility features. Our comprehensive solution features four key innovations: a professional 3D legal avatar with synchronized speech and lip movements, trilingual voice interaction supporting Arabic, French, and English, specialized Tunisian legal intelligence through AI agents trained on local frameworks, and real-time analytics dashboards tracking user engagement and system performance.

To realize this project, we followed the TDSP (Team Data Science Process) methodology through five sequential phases. We collected and structured over 72 official Tunisian legal PDFs, developed a sophisticated multi-agent RAG architecture featuring Azure AI Search with vector similarity search capabilities, and implemented hierarchical chunking preserving Tunisia's legal document structure. The AI processing pipeline utilizes Azure AI Foundry's dual-agent system with Question-Answer and Reviewer agents integrated with GPT-4o. Finally, we deployed a comprehensive React application with Azure Cognitive Services including Speech-to-Text, Text-to-Speech, and Animated Avatar services, featuring real-time communication, Voice Activity Detection, and Firebase authentication with automated CI/CD deployment through Azure Container Apps.

Key-words: Legal Artificial Intelligence, RAG (Retrieval-Augmented Generation), 3D Avatar, Multilingual Voice Interaction, Azure AI Services, TDSP (Team Data Science Process), LegalTech, Vector Search, Multi-Agent Systems, Tunisian Legal Framework.

ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454